

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ DỮ LIỆU TRONG MICROSERVICE

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Data Management - Choosing Right Database

Polyglot Persistence, Database-per-Service Pattern, Shared Database Anti-pattern

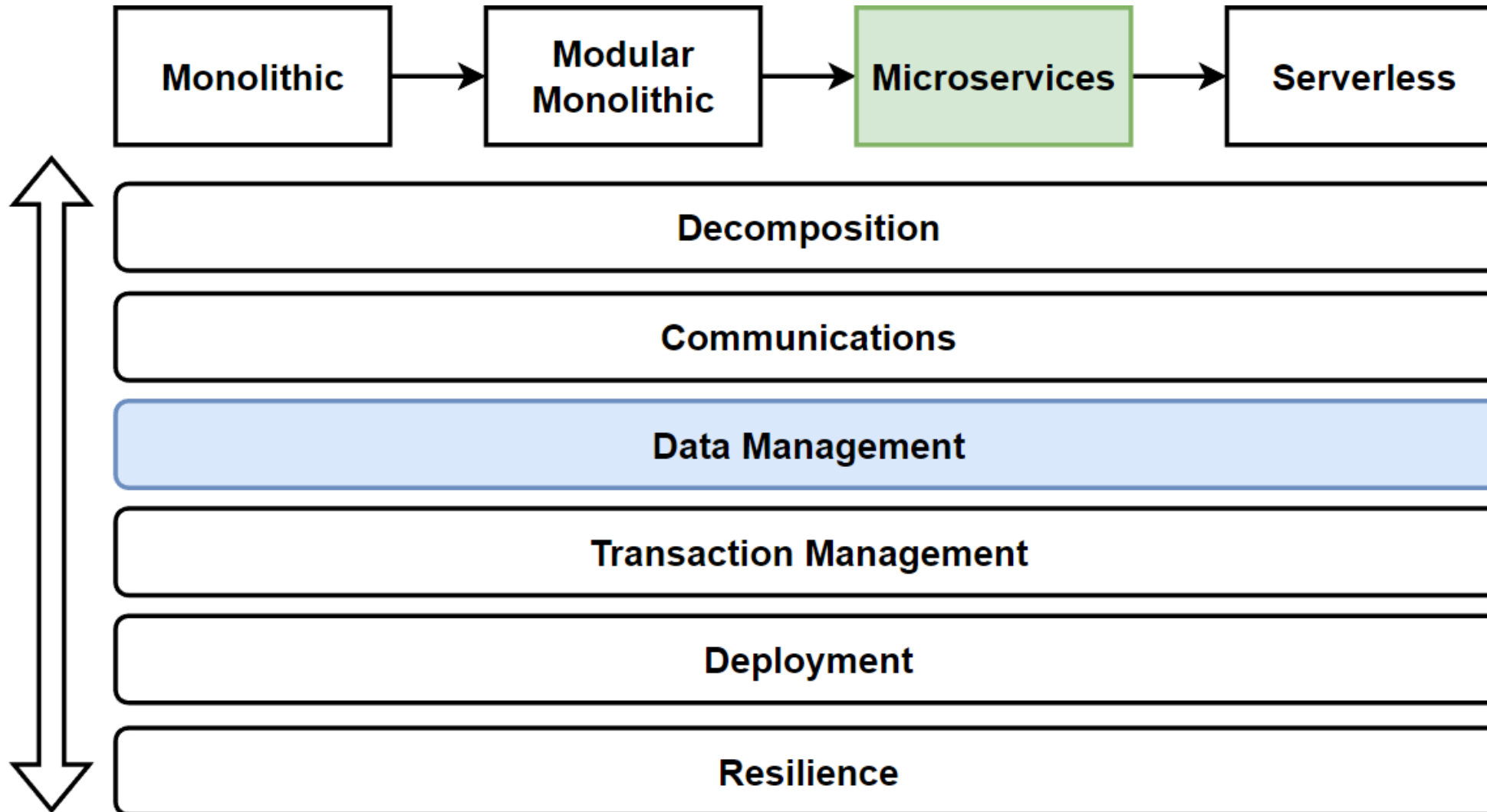
Relational and NoSQL Databases

CAP Theorem – Consistency, Availability, Partition Tolerance

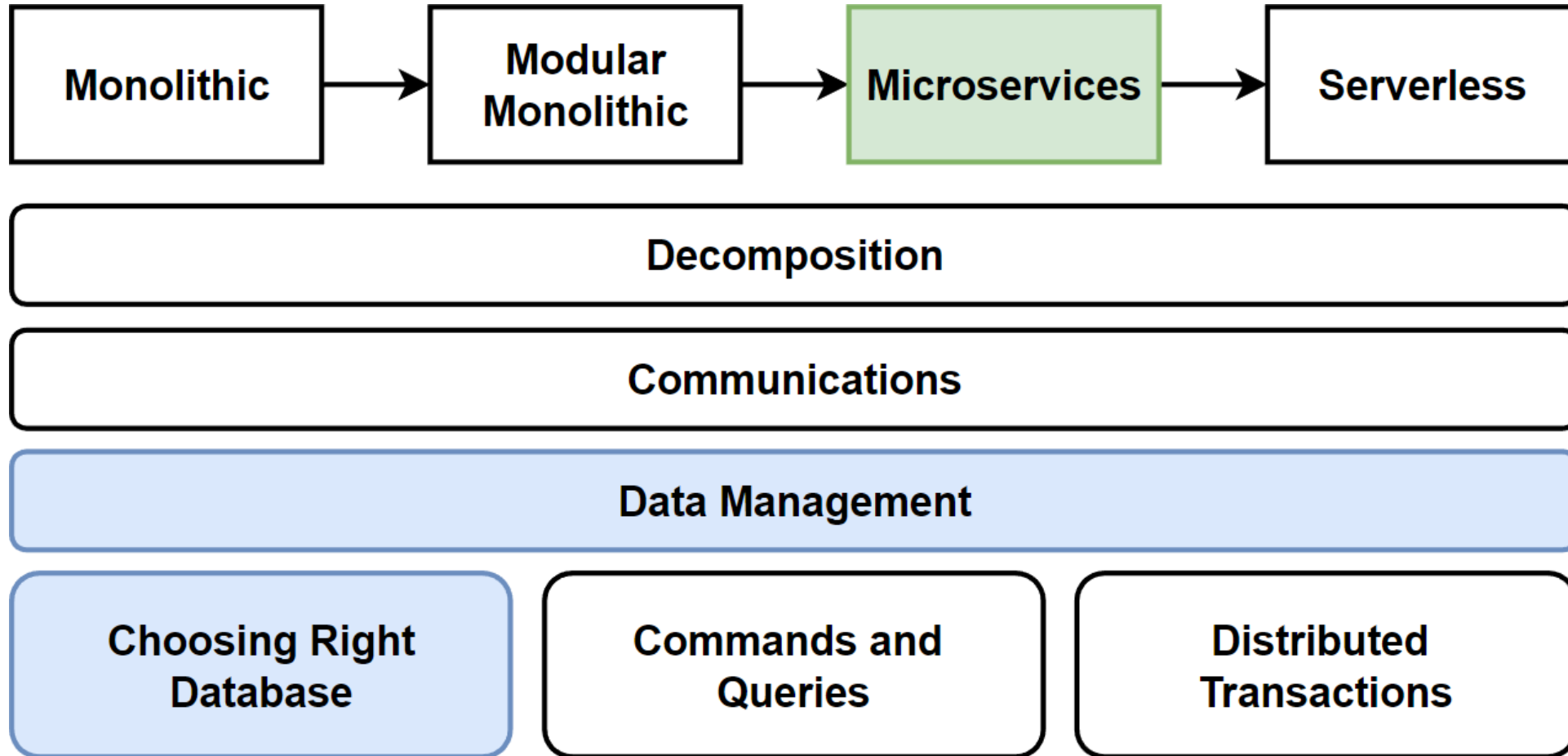
Data Partitioning: Horizontal, Vertical and Functional Data Partitioning

Database Sharding Pattern

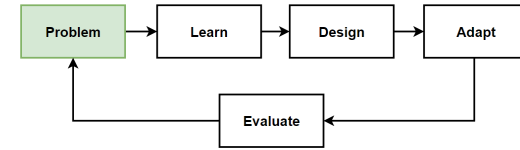
Architecture Design – Vertical Considerations



Microservices Data Management - Main Topics



Problem: Database Bottlenecks when Scaling

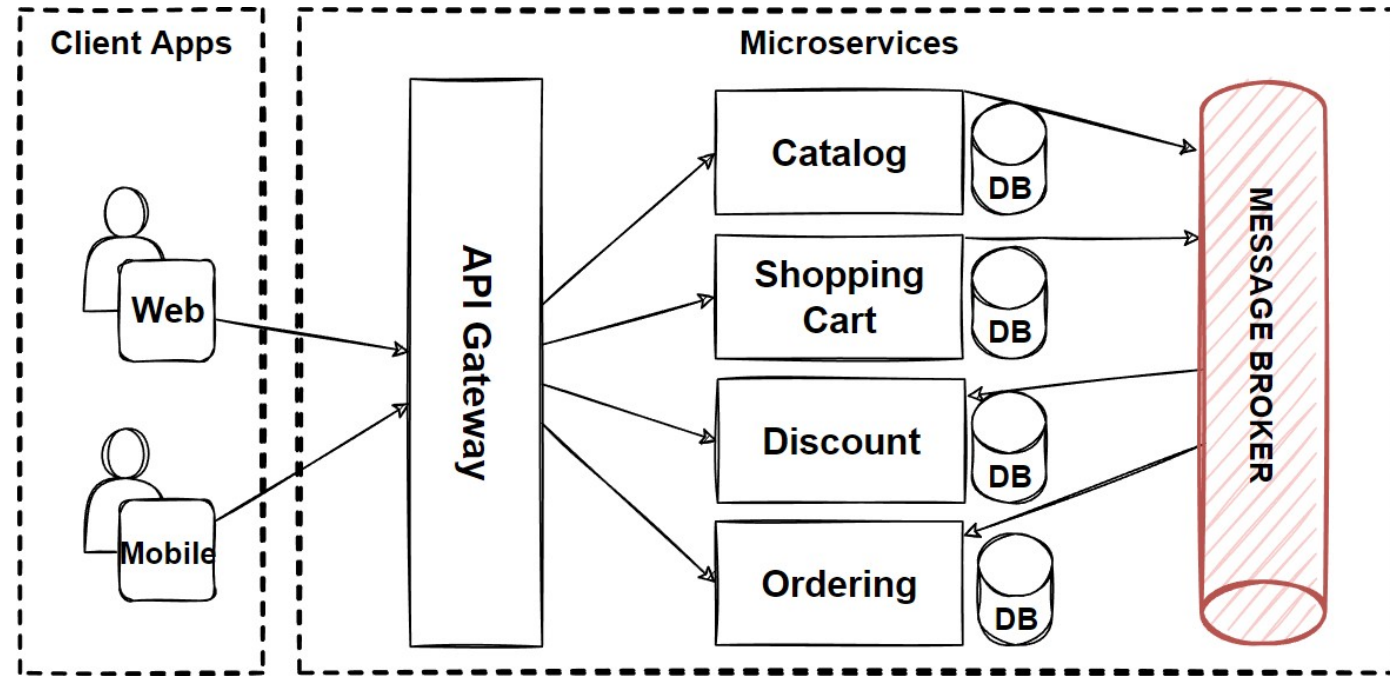


Problems

- Database are stateful service
- Scaling stateful services are not easy
- Vertical scaling has limits need to scale horizontally

Solutions

- Scale Stateful Application Horizontal Scaling
- Service and Data Partitioning along Business Boundaries - Shards/Pods
- Use NoSQL Database to gain easy partitioning features

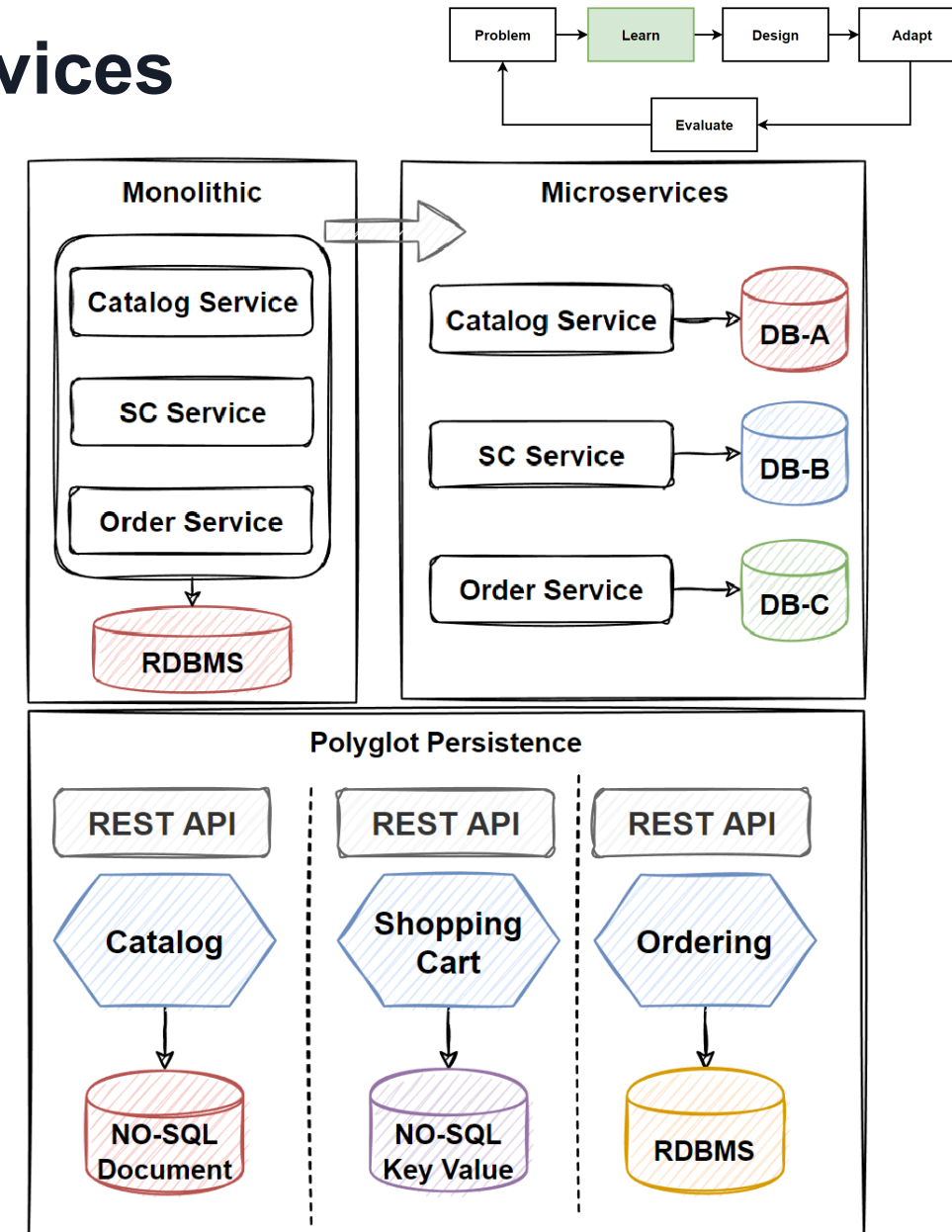


Question

- How to Choose a Database for Microservices ?

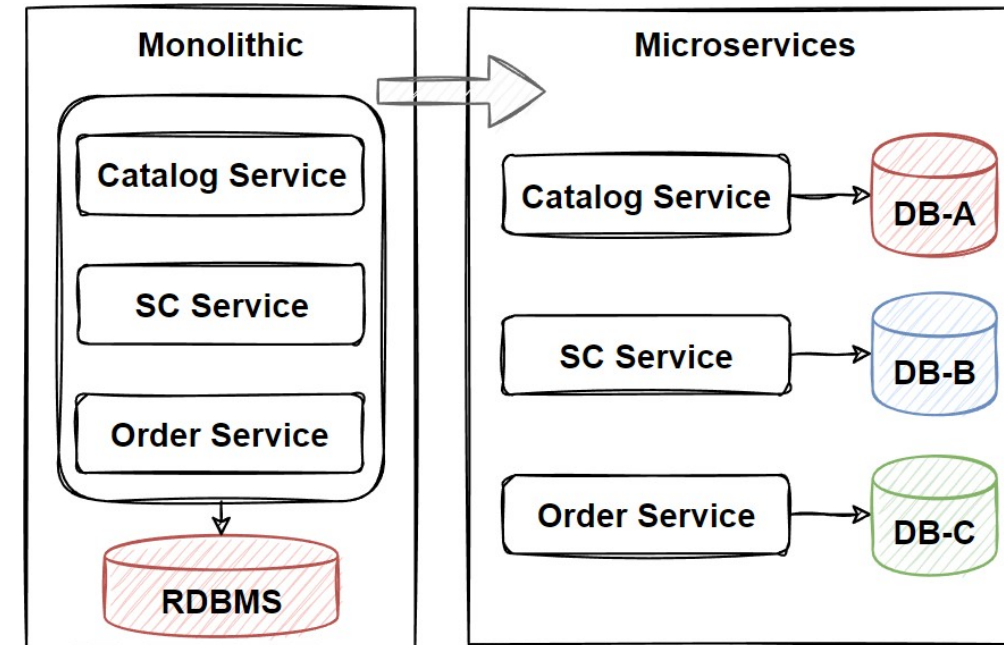
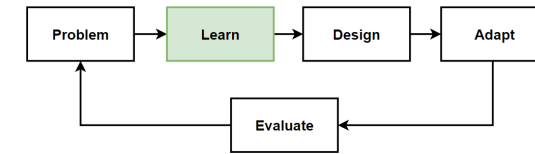
Polyglot Persistence Principle in Microservices

- Every microservice has its **own data**, the **data integrity** and **data consistency** should **consider** very **carefully**.
- Microservices should **not share database** with each other.
- Microservices **share data** over the application microservices with **using Rest APIs**. **Broke** the **unnecessary dependencies** between microservices.
- If there is an **update on 1 microservices database schema**, the update **shouldn't be directly affect** to other microservices.
- **Limit the scope of changes** on microservices when any database schema changes happened.
- We can **pick different databases** as per microservices which database can pick the best option: "**polyglot persistence**" principle.
- **Challenges:** Duplicated or partitioned data can make problems of data integrity and data consistency. Strict or Eventual consistency.



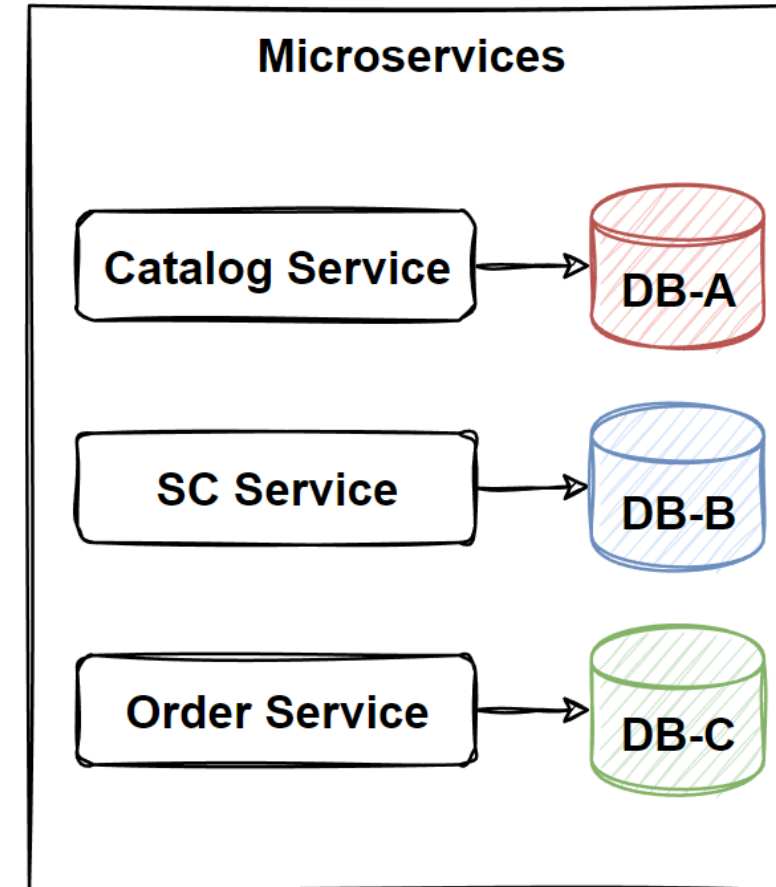
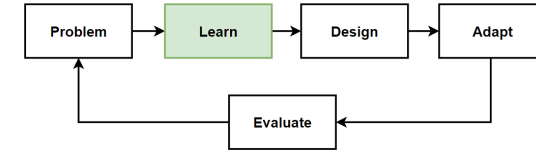
Polyglot Persistence Principle in Microservices-2

- In monolithic applications **can't be duplicate** and not be any data consistency problems, strict database transaction management.
- Microservices architecture, we have to **welcome duplicate datas** and **eventual consistency**.
- How we can **welcome duplication** and **un-consistent data** ?
- **Accept eventual consistency** data in microservices where it is possible.
- **Define** our **consistency requirements** before design the system.
- If need to **strong consistency** and **ACID** transactions, we should follow traditional approaches.
- Use **event driven architecture**: microservices publish an event when any changes happened on data model.
- Subscriber microservices **consume** and **process event** afterwards in eventual consistency model.
- Polyglot Persistence gives us scale independently and **avoid single-point-of-failure** of bottleneck databases.



Microservices Database Management Patterns and Principles

- **Managing** a microservices **database** is **challenging job**, should have a strategy.
- **ACID**: atomicity, consistency, isolation and durability.
- **Challenging to implement queries and transactions** that visits to several microservices.
- **Microservices Database Management Patterns and principles:**
 - The Database-per-Service pattern
 - The API Composition pattern
 - The CQRS pattern
 - The Event Sourcing pattern
 - The Saga pattern
 - The Shared Database anti-pattern



Microservices Database Management Patterns and Principles - Overview

- **The Database-per-Service Pattern**

In order to be a loose coupling of services, each microservice should have its own private database. Design database architecture for microservices.

- **The API Composition Pattern**

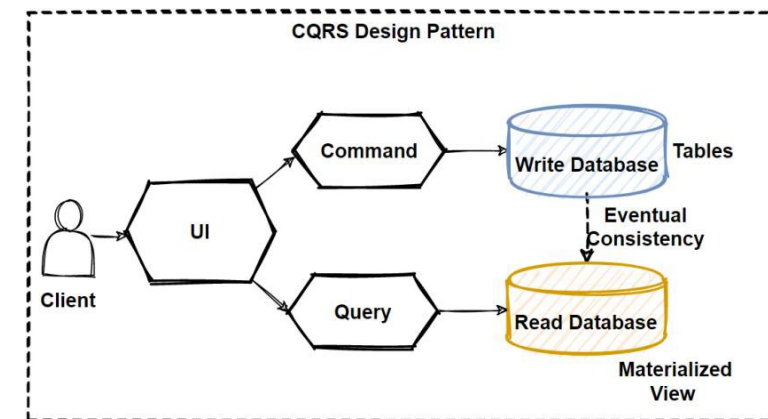
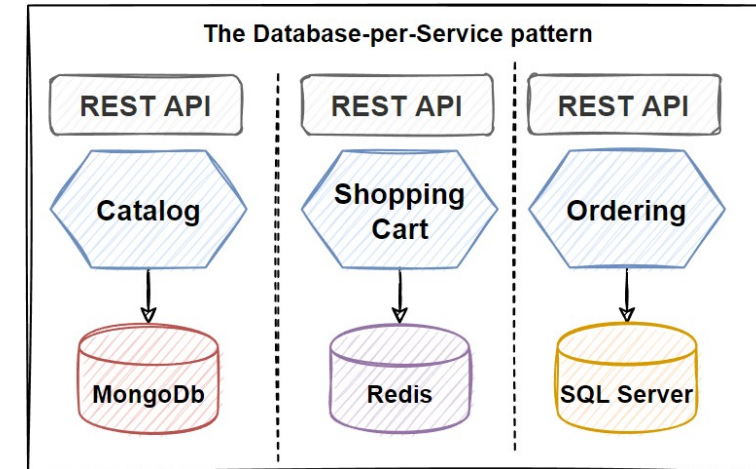
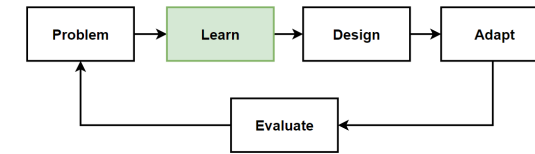
Retrieving data from several services also need a set of patterns and practices. When implements a query by invoking several microservices, we will follow the API Composition, Gateway Aggregation patterns for combining the results.

- **The CQRS Pattern**

The command query responsibility segregation (CQRS) is provide to seperate commands and queries database in order to better perform querying several microservices.

- **The Event Sourcing Pattern**

The Event Sourcing pattern basically provide to accumulate events and aggregates them into sequence of events in databases. We can replay at certain point of events.



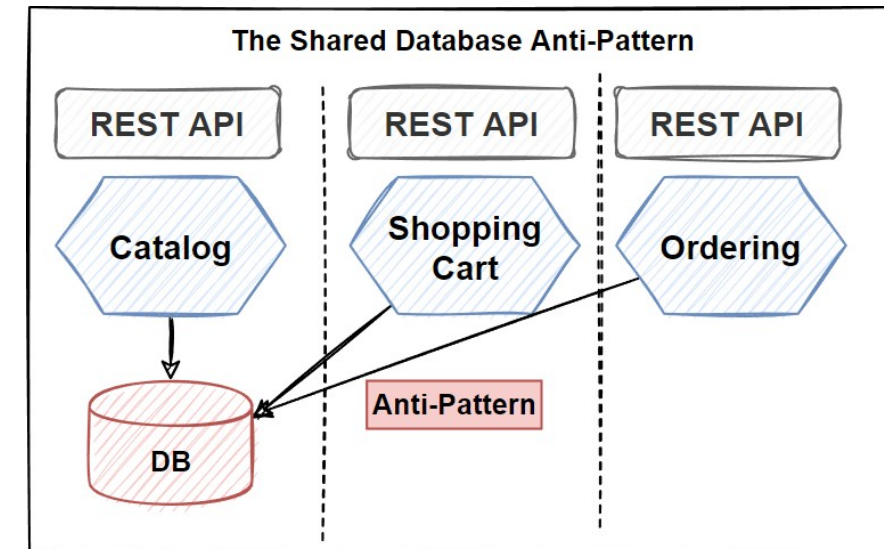
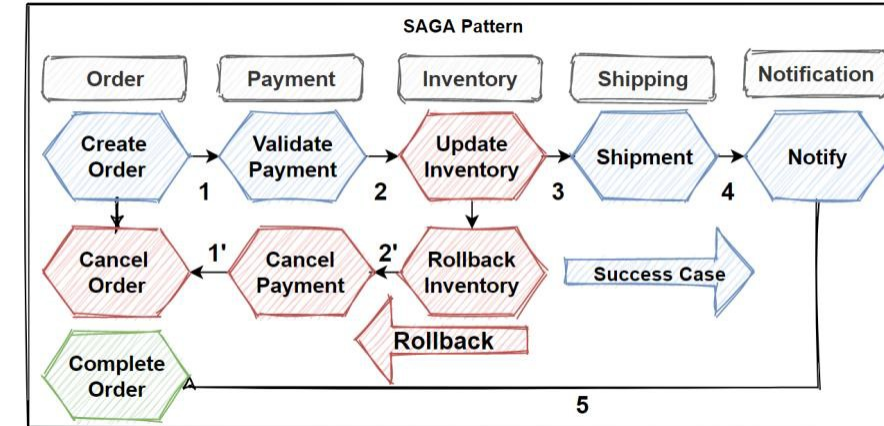
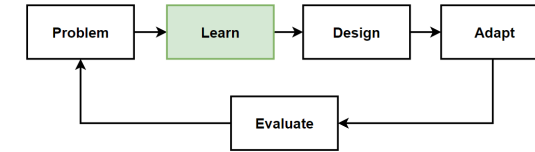
Microservices Database Management Patterns and Principles – Overview 2

- **The Saga Pattern**

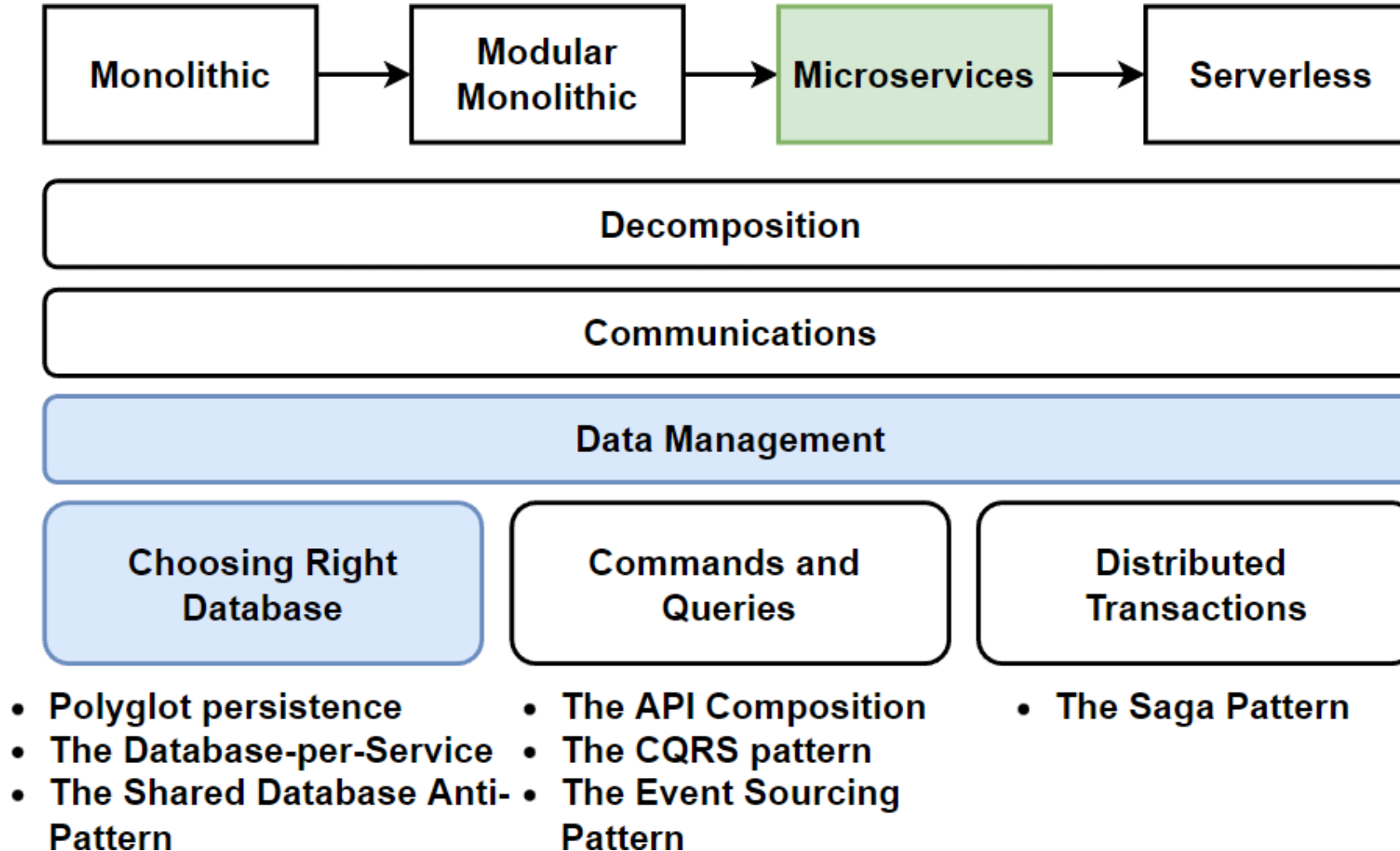
Transaction management is really hard when it comes to microservices architectures. We use Saga Pattern to implement transactions between several microservices and maintaining data consistency.

- **The Shared Database Anti-Pattern**

You can create a single shared database with each service accessing data using local ACID transactions. But it is against to microservices nature and will cause serious problems in the future of applications.

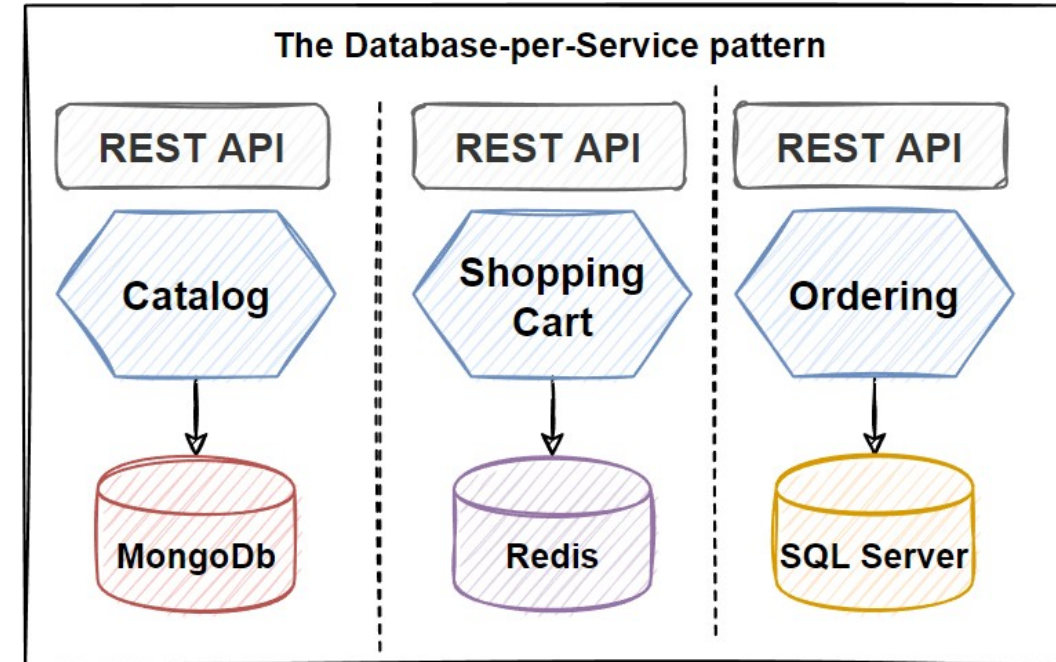
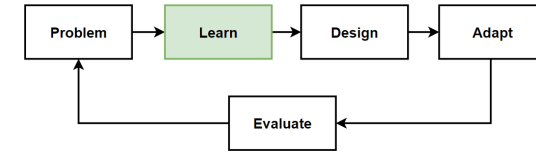


Microservices Data Management – Patterns and Principles

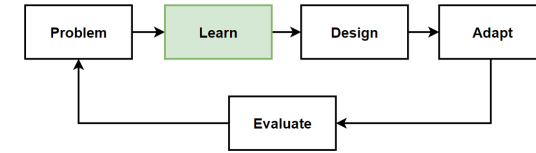


The Database-per-Service Pattern

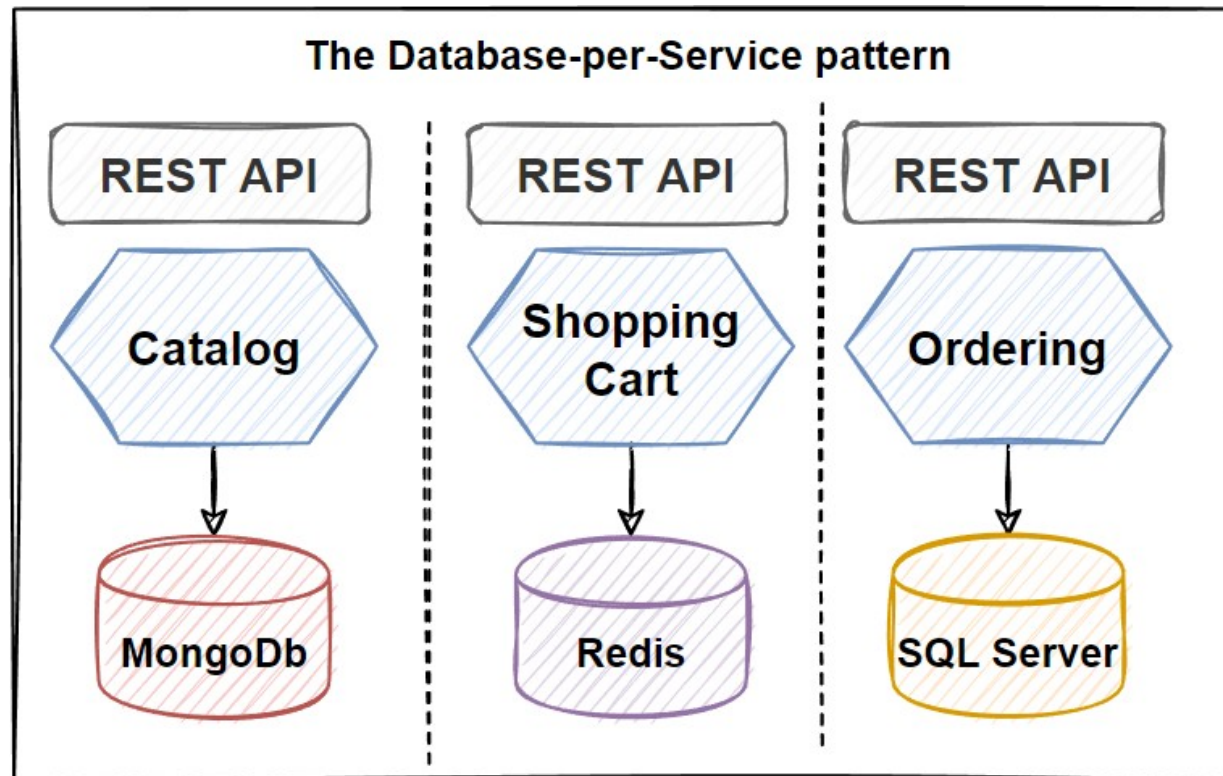
- Microservices should be **loosely coupled, scalable, and independent**. When shifting to the monolithic architecture to microservices architecture, should **decomposes databases**.
- **Distributed data model** with many **smaller databases** for particular microservice.
- **Data schema changes** can perform **without any impact** on other microservices, Changes to an individual database don't impact other services.
- There isn't a **single point of failure** in the application, the application is becomes more resilient.
- **Individual databases** are **easier to scale**, if 1 service peak the requests that only that microservice can scale independently.



The Database-per-Service Pattern-2

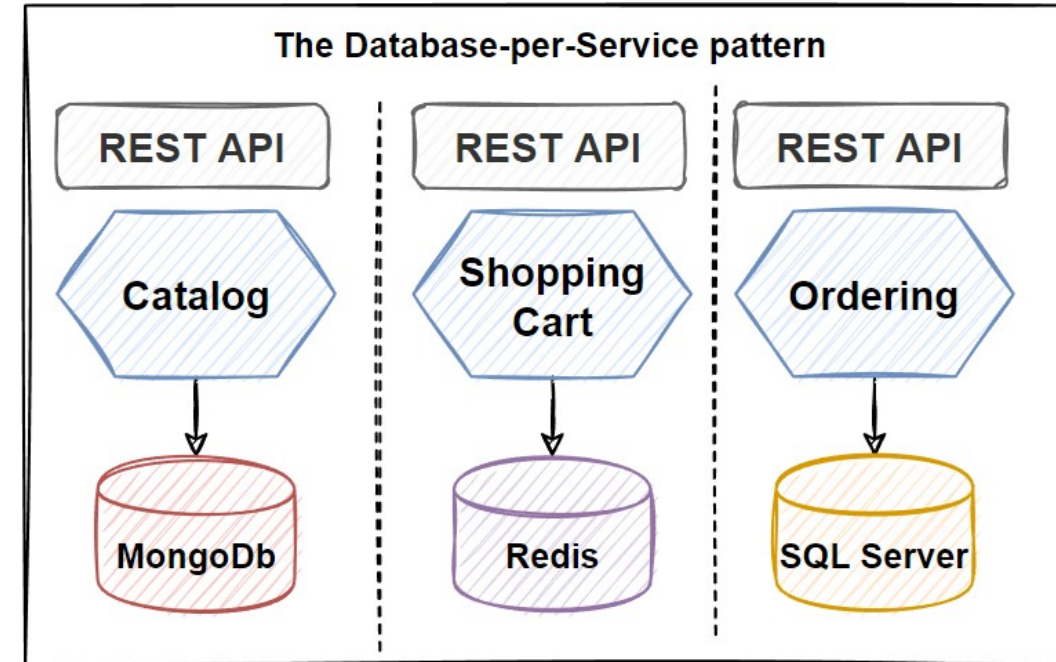
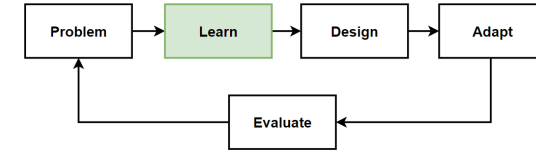


- **Separating databases** can give us the ability to **pick the best optimized database** for our microservices.
- Can choose to include **relational, document, key-value**, and even **graph-based** data stores.
- Using the **most efficient database** depending on the service requirements and functionality.



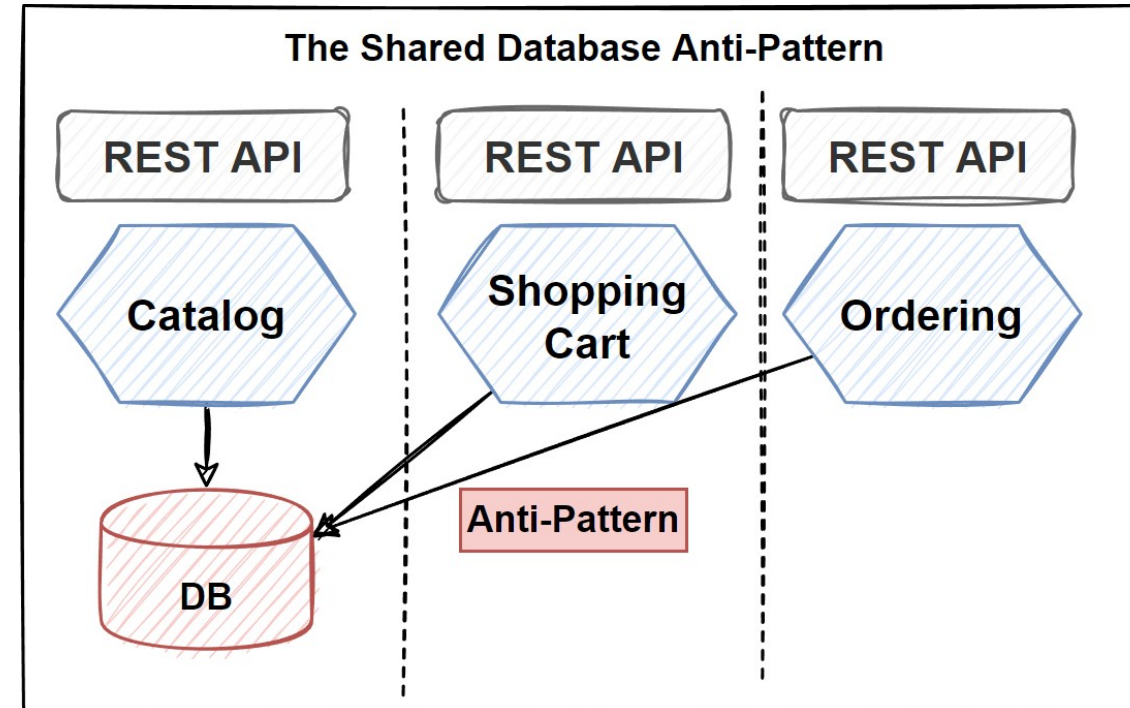
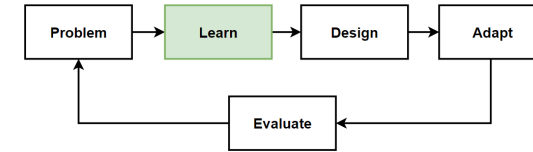
Drawbacks of Database-per-Service Pattern

- Services need a communication method to **exchange data, inter-service communication**.
- Each service **must provide a clear API**, that need to make resilience of these communications like applying **retry** and **circuit breaker patterns**.
- **Distributed transactions** across microservices can negatively **impact consistency** and **atomicity**.
- **Complex queries**, no simple way to execute join queries on multiple data stores.

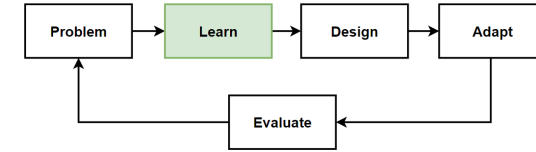


The Shared Database Anti-Pattern

- If we **don't follow Database-per-Service** pattern and use **Shared Database** for several microservices = **Anti-Pattern**
- It is **against to microservices nature** and will cause serious problems in the future of applications.
- When using a **shared database**, the microservices **lose** their **core properties**: scalability, resilience, and independence.
- You will face to **develop big a few monolithic applications** instead of microservices.
- Shared database can **block microservices** due to **single-point-of-failure**.
- If shared database seems to be the best option for the microservices project, **should re-think** that we really need the microservices, the monolith would be the better choice.



The Shared Database Anti-Pattern - Pros and Cons

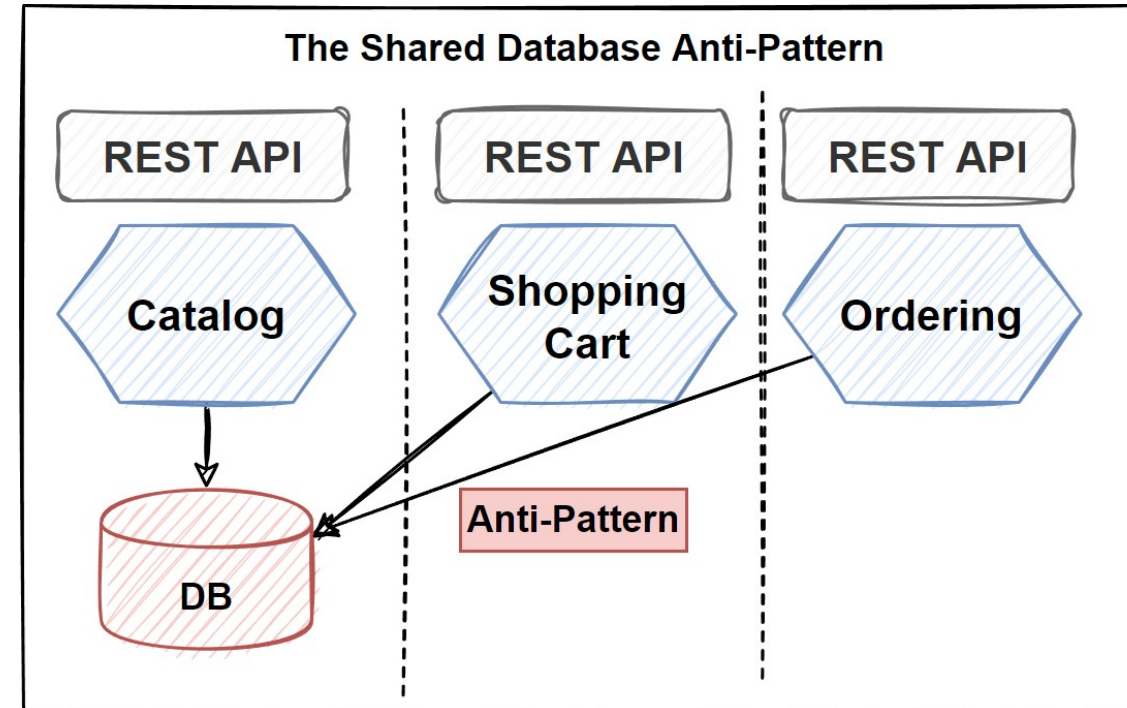


Benefits

- Transaction management, that we don't need to span the transactions over the microservices.
- Decrease duplicate data. Since data is fully constrained, we can easily execute complicated queries with joins.
- Able to follow ACID. Consistency of data and state easily manage when process fails.

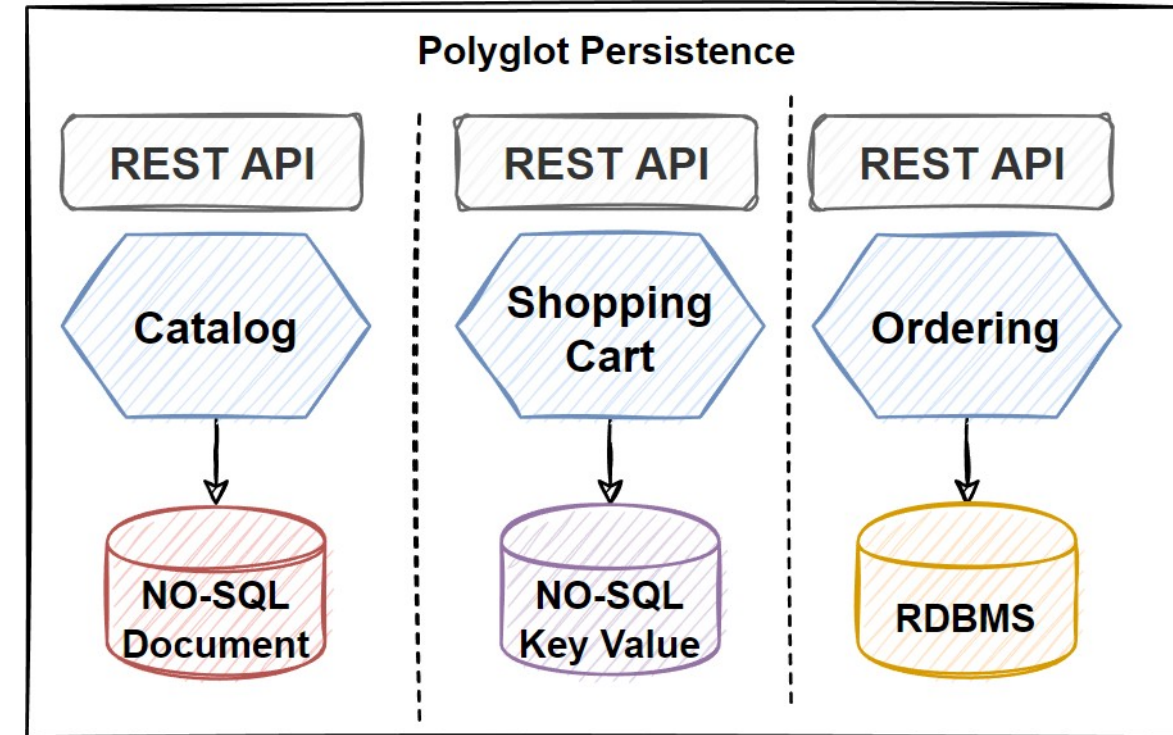
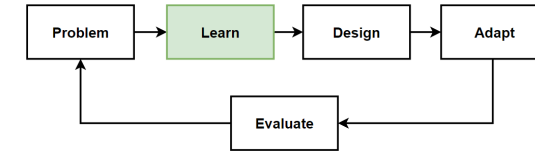
Drawbacks

- Microservices with shared databases can't easily scale.
- Shared database will become a single point of failure.
- Microservices won't be independent in terms of development and deployment.

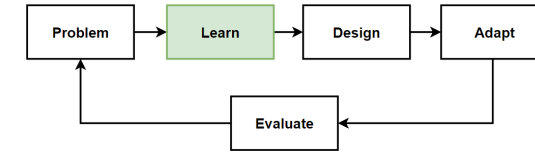


Polyglot Persistence

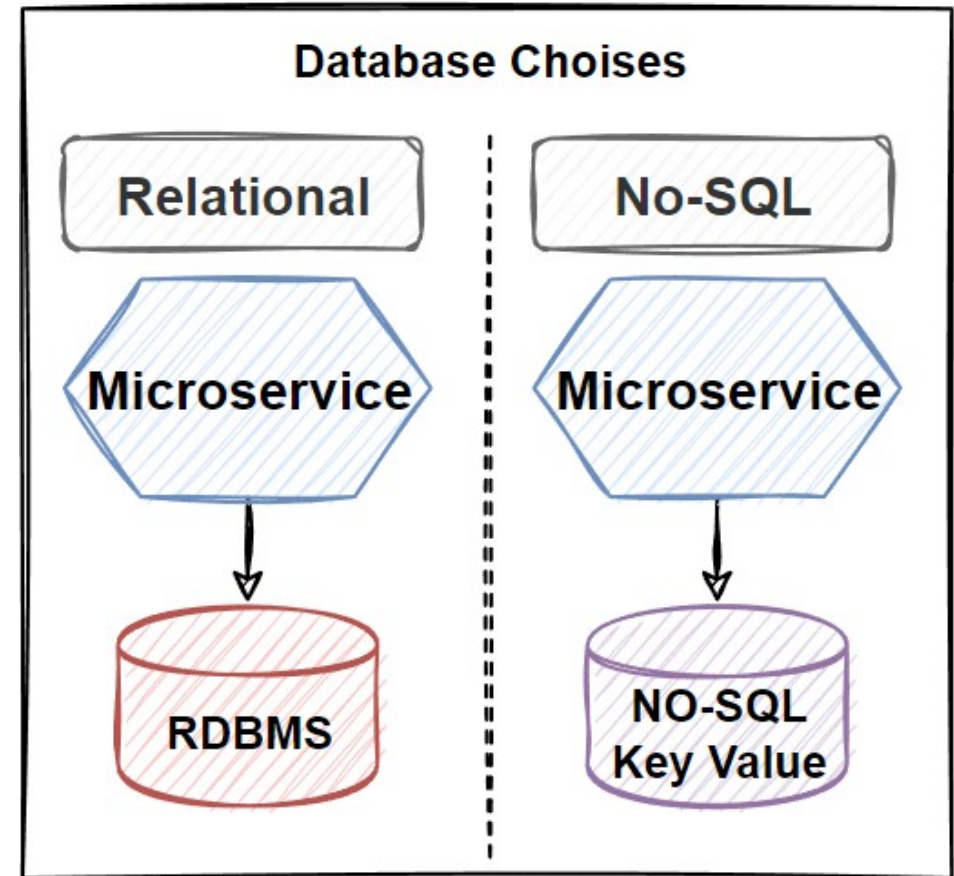
- Microservices enables using **different kinds of data storing** technologies.
- Each development team can **choose the persistence technology** that suits the needs of their service best.
- Martin Fowler - **Polyglot persistence** will **come with a cost** - but it will come because the benefits are worth it.
- When **relational databases** are **used inappropriately**, they give damaged on application development.
- Example of **looked up page elements by ID**, much better suited to a key-value No-SQL databases than relational databases.
- **How to Choose a Database for Microservices ?**



Database Choices - Relational and NoSQL Databases

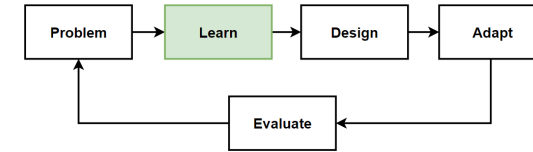


- **Relational** and **NoSQL** are **two types** of database systems commonly implemented in microservices architecture.
- New approach of databases which is called **NewSQL**.
- **Relational** and **NoSQL** are different in terms of the **storing data**, and **accessing data**.

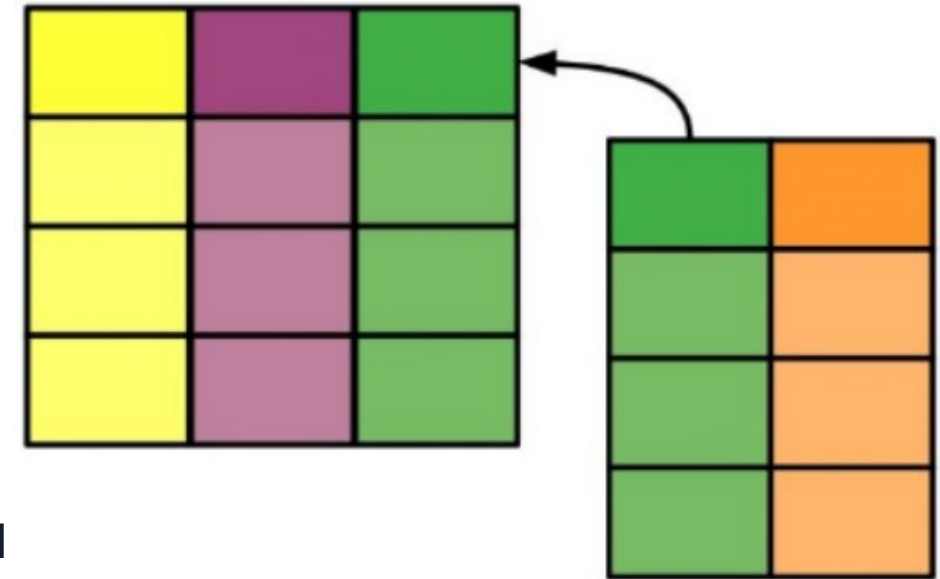


Relational Databases - RDBMS

- **Relational databases** provides storing data into related data tables.
- Relational database tables have a **fixed schema**, use SQL to manage data and support transactions with **ACID principles**.
- A table uses **columns** and **rows** for storing the actual data. Each table will have a column that must have **unique values**—known as the **primary key**.
- When one **table's primary key** is used in **another table**, this column in the second table is known as the **foreign key**.
- On microservices, we should **not consider** using **single big relational database**.
- The main advantages of Relational database is **ACID compliance**. If one change fails, the **whole transaction will fail**.
- Polyglot persistence in microservices, still **relational databases** is **good option** some certain type of problems.
- **Example of relational databases** are Oracle, MS SQL Server, MySQL, PostgreSQL.



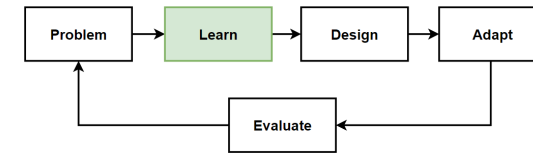
Relational



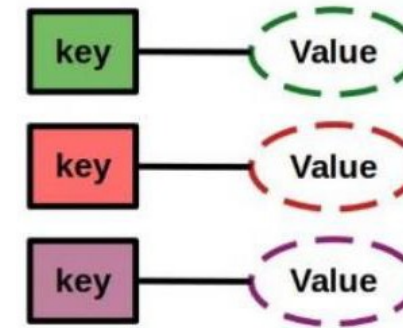
SQL

No-SQL Databases (Non-Relational Databases)

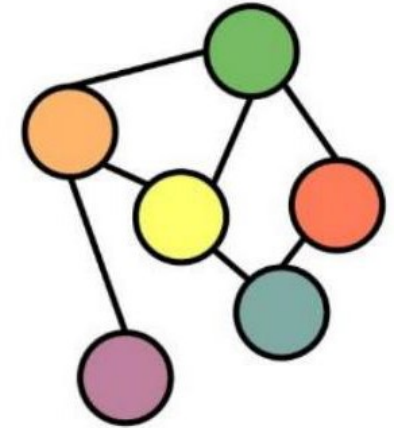
- **No-SQL databases** has different types of stored data and data models: **Document**, **Key-value**, **Graph-based**, **Column-based** databases.
- **Ease-of-use, scalability, resilience, and availability** characteristics.
- NoSQL databases stores **unstructured data**, and this gives huge **performance advantage**.
- **NoSQL stored unstructured data** in key-value pairs or JSON documents.
- No-SQL databases **don't provide ACID** guarantees.
- **Drawback** is transaction management.



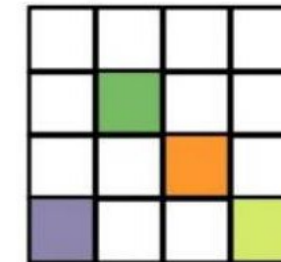
Key-Value



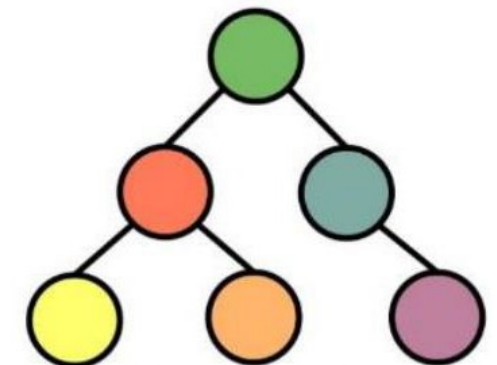
Graph



NoSQL



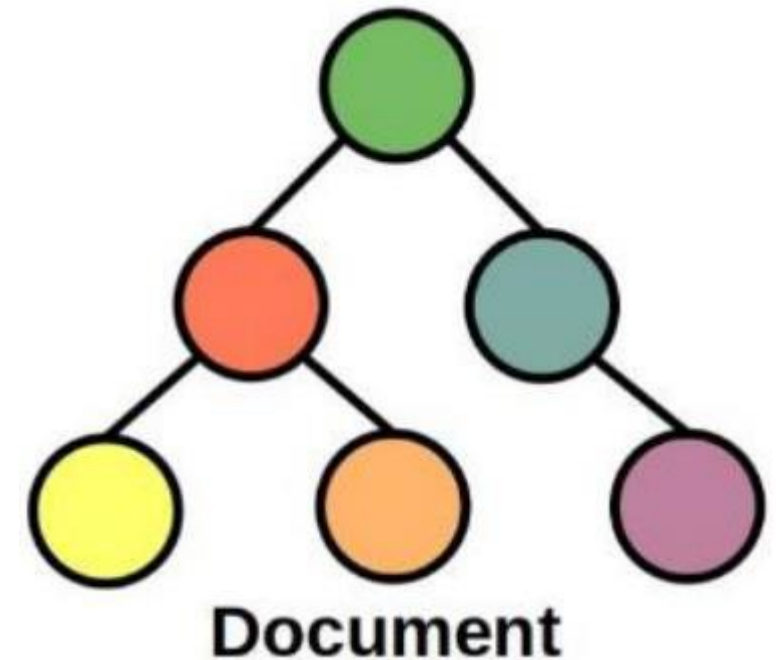
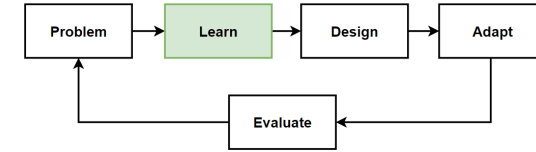
Wide
Column



Document

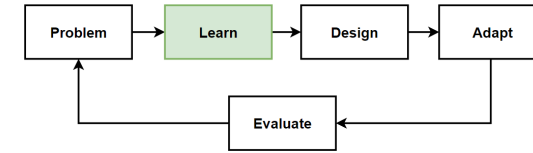
No-SQL Document Databases

- **Document databases** stores and query data in JSON-based documents.
- **Data** and **metadata** are **stored hierarchically**.
- **Objects** are **mapping** to the **application code**.
- **Don't have to run JOINS** or decompose data across tables.
- **Scalability**, document databases can **distributed very well**.
- Best choice for **content management** and **storing catalogs**.
- I.e. **products** data can store in **document database** for **e-commerce applications**.
- **Example Document Databases**: MongoDB and Cloudant.

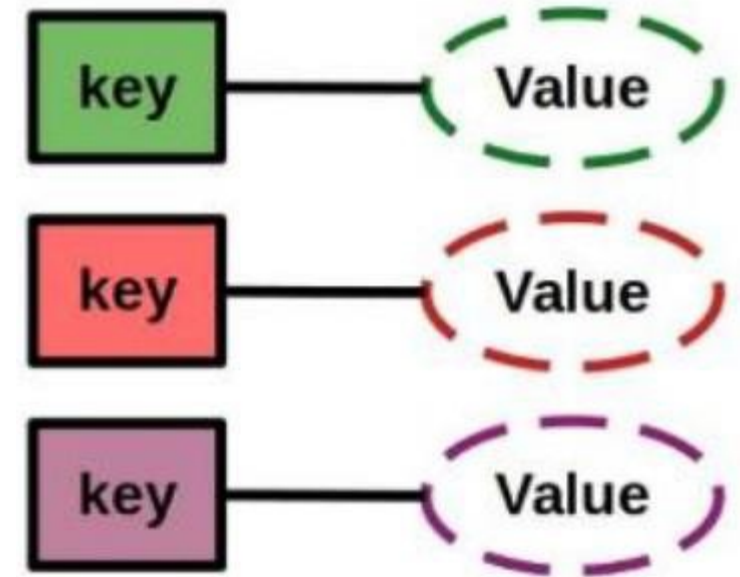


No-SQL Key-Value Databases

- **Data is storing** as a collection of **key-value pairs** in Key-value **NoSQL database**.
- Data is represented as a **group of key-value** in the database.
- **Best choice** for **session-oriented** applications.
- I.e. storing **customer basket data** into **key-value database**.
- **Example Key-Value Databases:** Redis, Amazon DynamoDB, Azure CosmosDB, Oracle NoSQL Database.

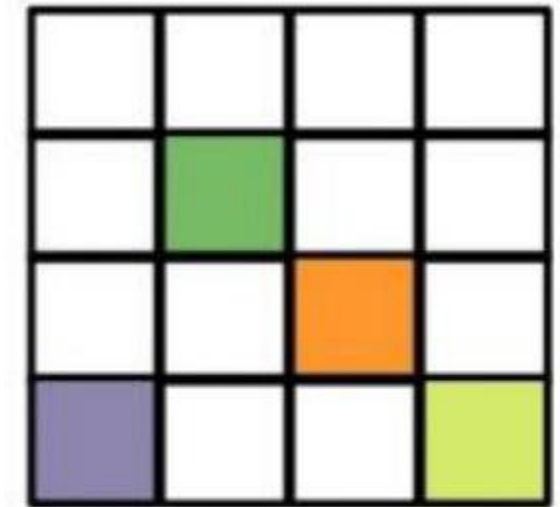
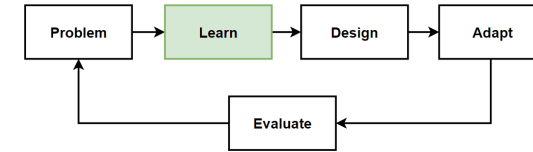


Key-Value



No-SQL Column-Based Databases

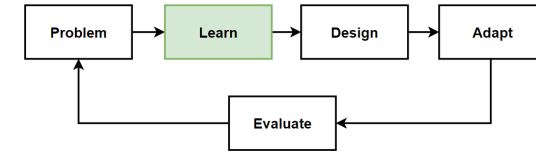
- **Column-based databases** also known Wide-Column Databases.
- **Data is stored in columns**, by this way, it can access necessary data **more faster** than if we compare to storing data in rows.
- If you **select mostly same columns** in your databases, its good to use this databases.
- It **doesn't scanning the unnecessary information** in a whole row.
- **Column-based databases** can **scale by columns independently**.
- **Columns** could be **different database servers**.
- I.e. building a **Data warehouse, Big Data processing**.
- **Apache Cassandra, Apache HBase or Amazon DynamoDB, Azure CosmosDB.**



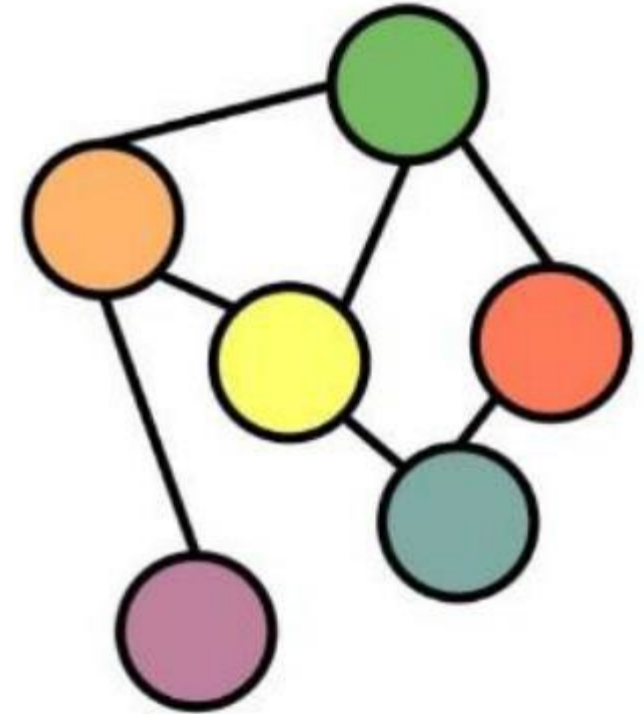
**Wide
Column**

No-SQL Graph-Based Databases

- **Graph-based databases** is stores data in a **graph structure** into **node**, **edge**, and **data properties**.
- **Data entities** are **connected** in **nodes**.
- The main benefit of a **graph-based databases** is to **store** and **navigate graph relationships**.
- I.e. **fraud detection**, **social networks**, and **recommendation engines**.
- **Example of Graph-based databases** are OrientDB, Neo4j, and Amazon Neptune.



Graph



When to Use Relational Databases ?

- **ACID compliance, Data Consistency**

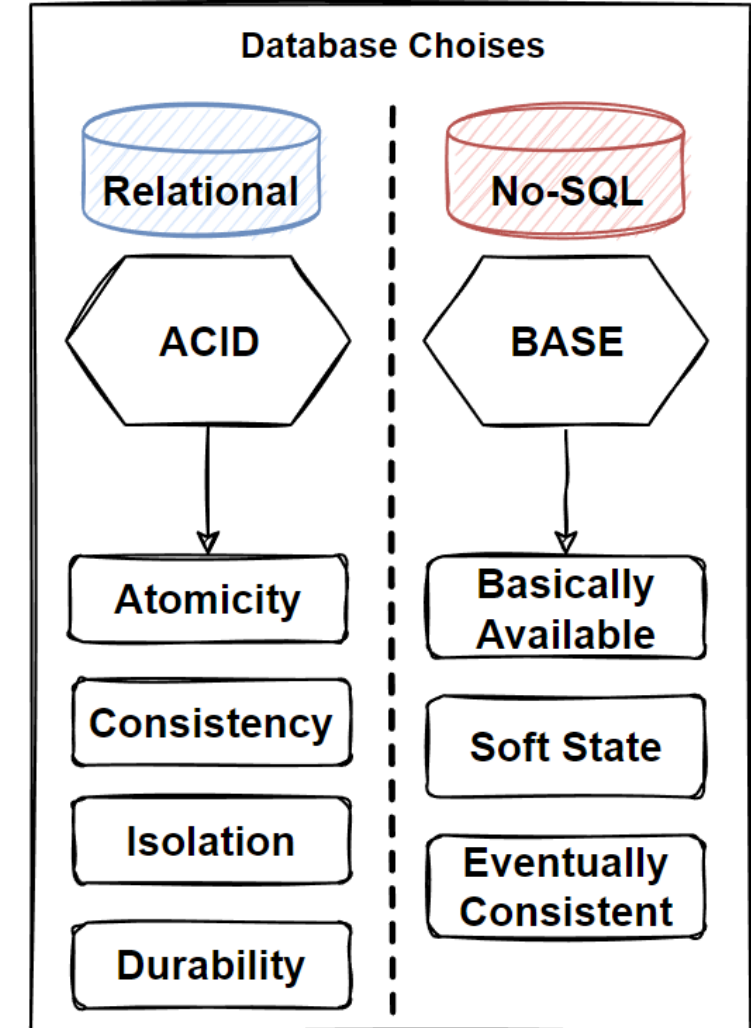
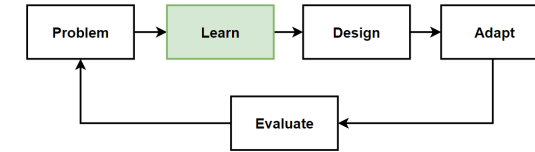
Relational database is ACID compliance. ACID - Atomicity, Consistency, Isolation, and Durability (ACID) guarantees the reliability of database transactions.

- If one change fails on database, database should remain in the previous state that was before the transaction.
- Data Consistency is fully provided, if you need strong consistency, good to choose Relational Databases, supports complex transactions.

- **Predictable Data, Low Workload Volume**

If application data is predictable, as per structure, size, and frequency of access, within thousands of transactions per second, relational databases are still the best choice.

- Normalization also reduce the size of the data on disk by limiting duplicate data and anomalies.



When to Use Relational Databases ? - 2

- **Read Requirements, Complex Join Queries**

Relational Database has a fixed schema. When relationships between tables are important and data is highly structured and requires referential integrity.

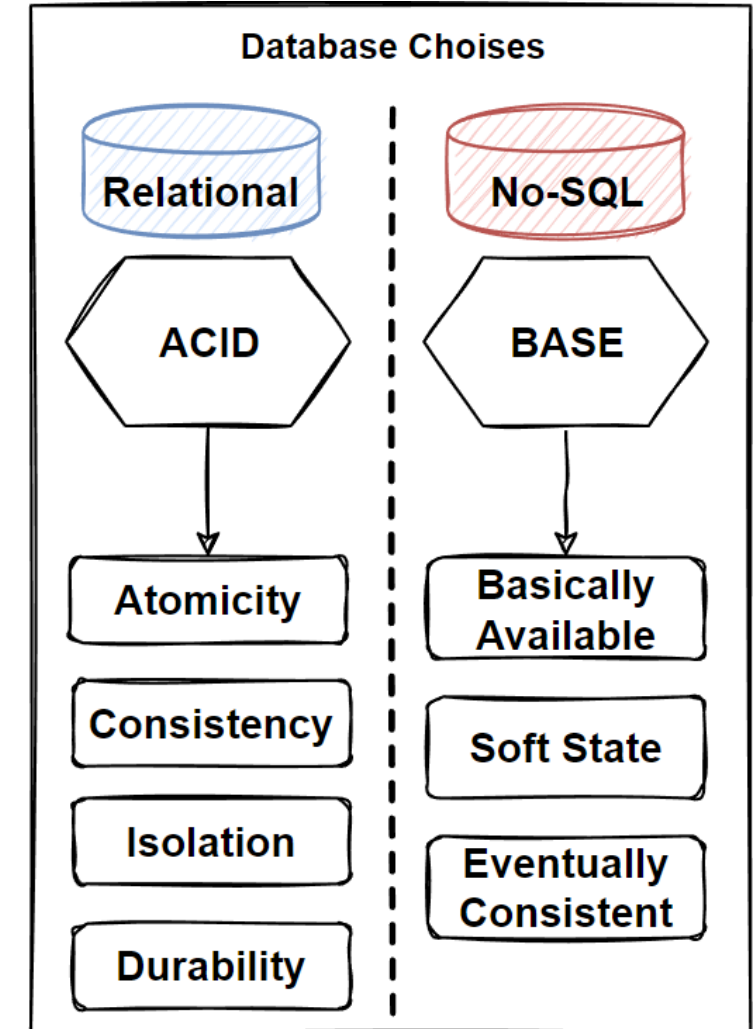
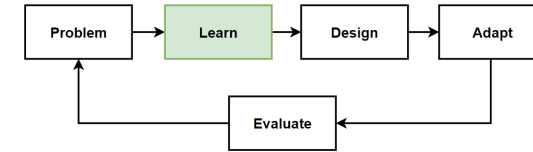
- Can work with complex queries, table joins and reports on normalized data models.

- Relational Database supports a powerful SQL query language.

- **Deployments, Centralized Structure**

Relational Databases will be deployed to large and one or few locations. Relational database has centralized structure.

- Relational databases have a single point of failure with failover.
- Relation database is deployed in vertical fashion.



When to Use No-SQL Databases ?

- **Flexible Schema, Dynamic Data**

NoSQL Database has no fixed schema. Allows to add or remove attributes into their model with dynamically. When your data is dynamic and frequently changes.

- Use case of implement an IoT platform that stores data from different kinds of sensors with frequently changed the attributes of your data.

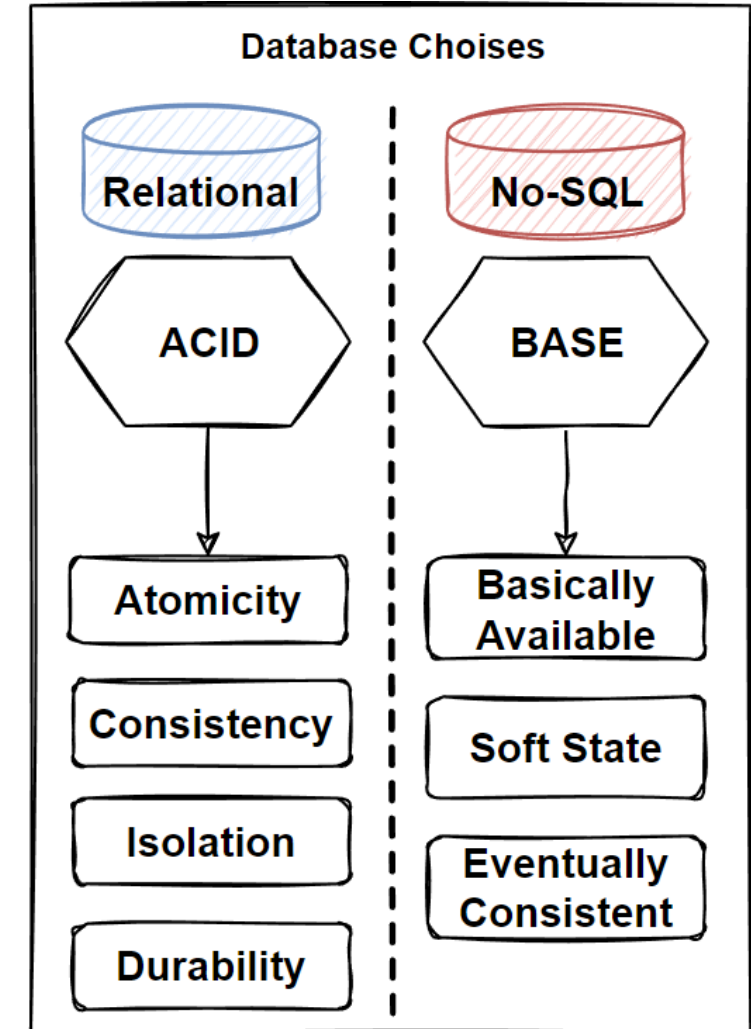
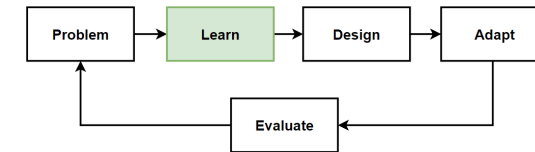
- **Un-predictable Data, High Workload Volume**

When you have high volume workloads and needs to horizontal scale with low latency. NoSQL databases have been designed for the cloud that naturally good for horizontal scaling.

- NoSQL Databases prioritize partition tolerance that designed for handling large amount of data or data coming in high velocity.

- **Frequently Change Data and Read Requirements**

When data is dynamic and frequently changes and Relationships are de-normalized data models and Data retrieve operations are simple and performs without table joins.



When to Use No-SQL Databases ? - 2

- **Data Consistency, BASE Model - Basically Available, Soft State, Eventually Consistent**

NoSQL Database is only eventually consistent and don't support transactions, focus on high volume data and horizontal scaling.

- **Write Performance Requirements**

NoSQL Database compromise consistency to achieve fast write performance, offers to fast write operations with Eventual consistency.

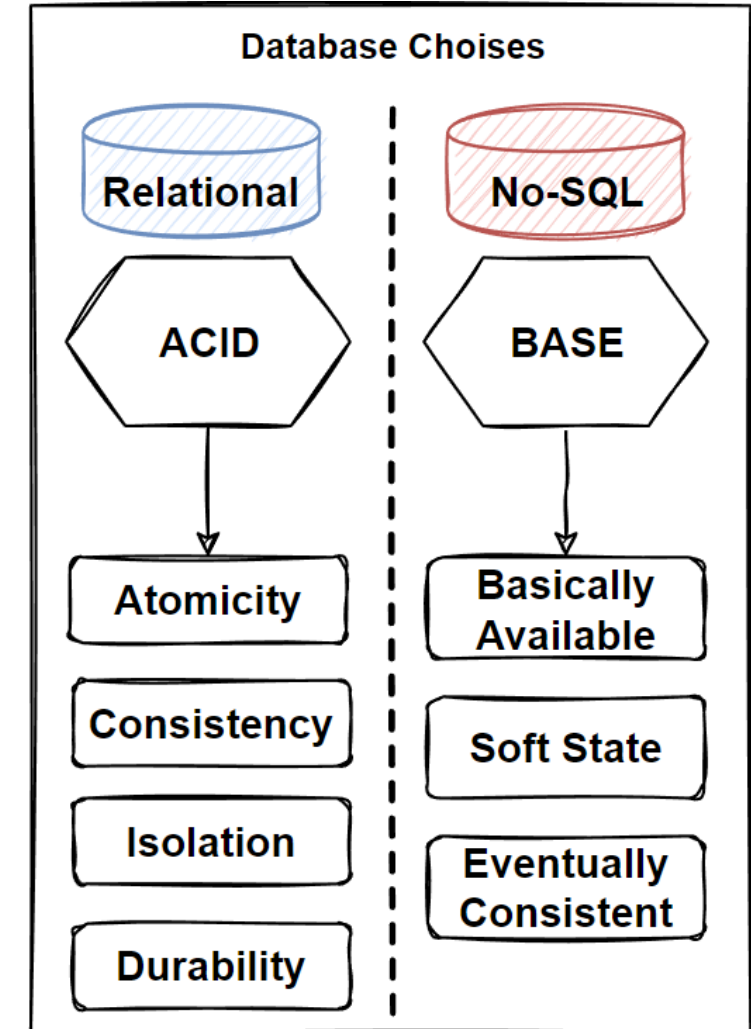
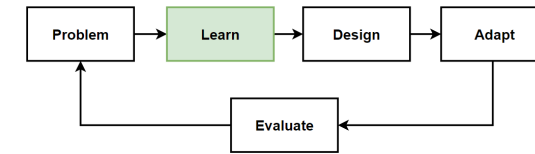
- **Not Good for Complex Join Queries**

NoSQL Databases perform best when data is stored in the same format not require to relation and join operations.

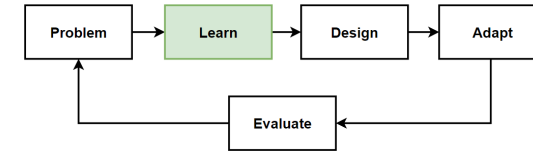
- **Deployments, De-centralized Structure**

NoSQL is scale horizontally so data is replicated across different geographical zones and provides better control over consistency, availability, and performance.

- NoSQL databases have no single point of failure, has decentralized structure, gives both read and write scalability, deployed in horizontally.



Best Practices When Choosing Data Store



- Use Right Tool for Right Job, Use the best data store for your data.

- **Don't use Relational Database Everywhere**

If using Relational Database for all microservices, would probably going wrong way, consider other data stores to data requirements.

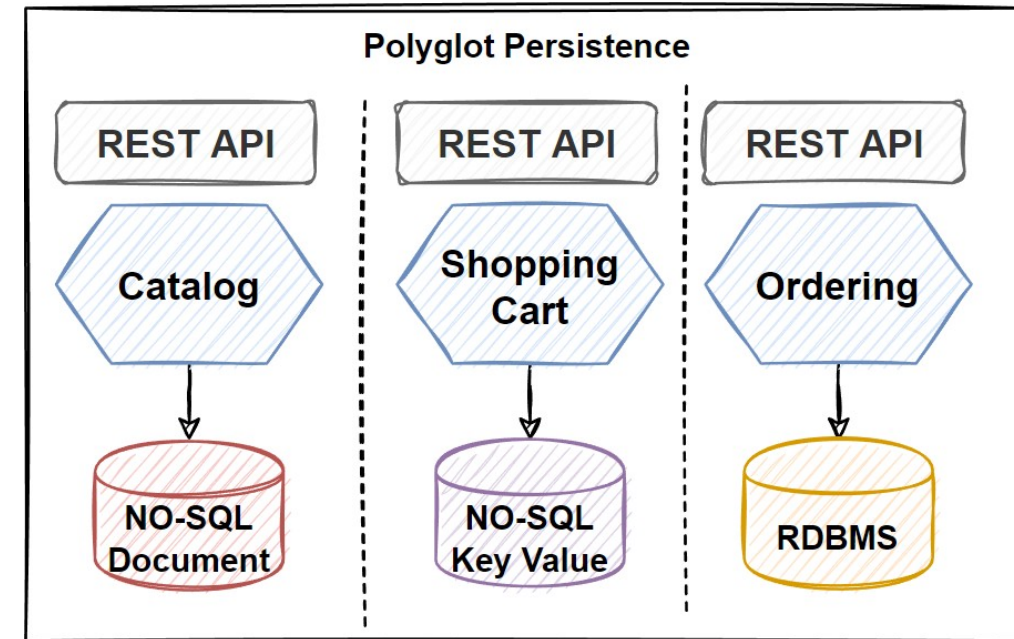
- **Don't use Single Data Store Technology, Differentiate**

Choose Alternatives to relational databases; Key/value stores, Document, Search engine, Time series, Column family, Graph databases.

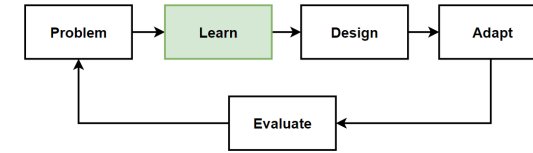
- **Focus The Data Type That Need to Store**

Consider the type of data that you have.

- Store JSON documents in No-SQL Document database.
- Put transactional data into a Relational SQL database.
- Use a time series data base for telemetry databases.
- Choose Blob Data Storage for blob datas.
- Put application logs into Elastic Search Databases.



Best Practices When Choosing Data Store - 2



- **Trade-offs between Availability and Consistency**

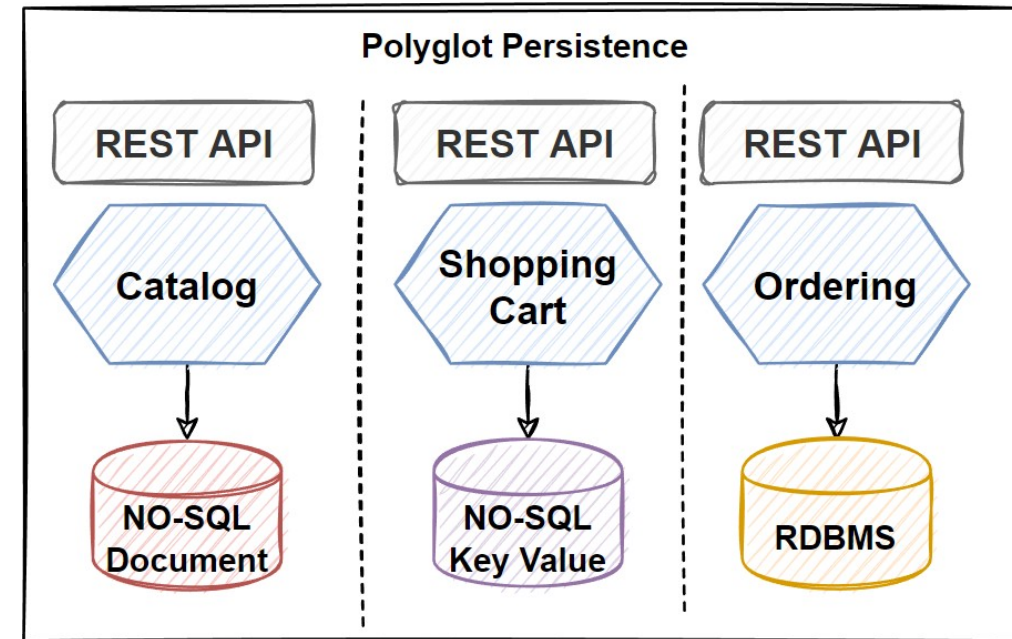
Understand the Trade-offs between Availability and Consistency. Should prefer High Availability over strong consistency as soon as possible to scale horizontally. (the CAP theorem)

- **Transactional Boundaries Between Microservices**

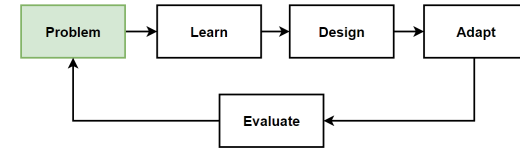
Consider business Transactional Boundaries Between Microservices, that data need to be consistent across those microservices. Prepare for compensating transactions in case of fail.

- **Competence of Development Teams**

Consider your team competences about database technologies. Skill set of your development team should cover optimize queries and tune for performance improvements.



Problem: Database Bottlenecks when Scaling, Different Data Requirements For Microservices

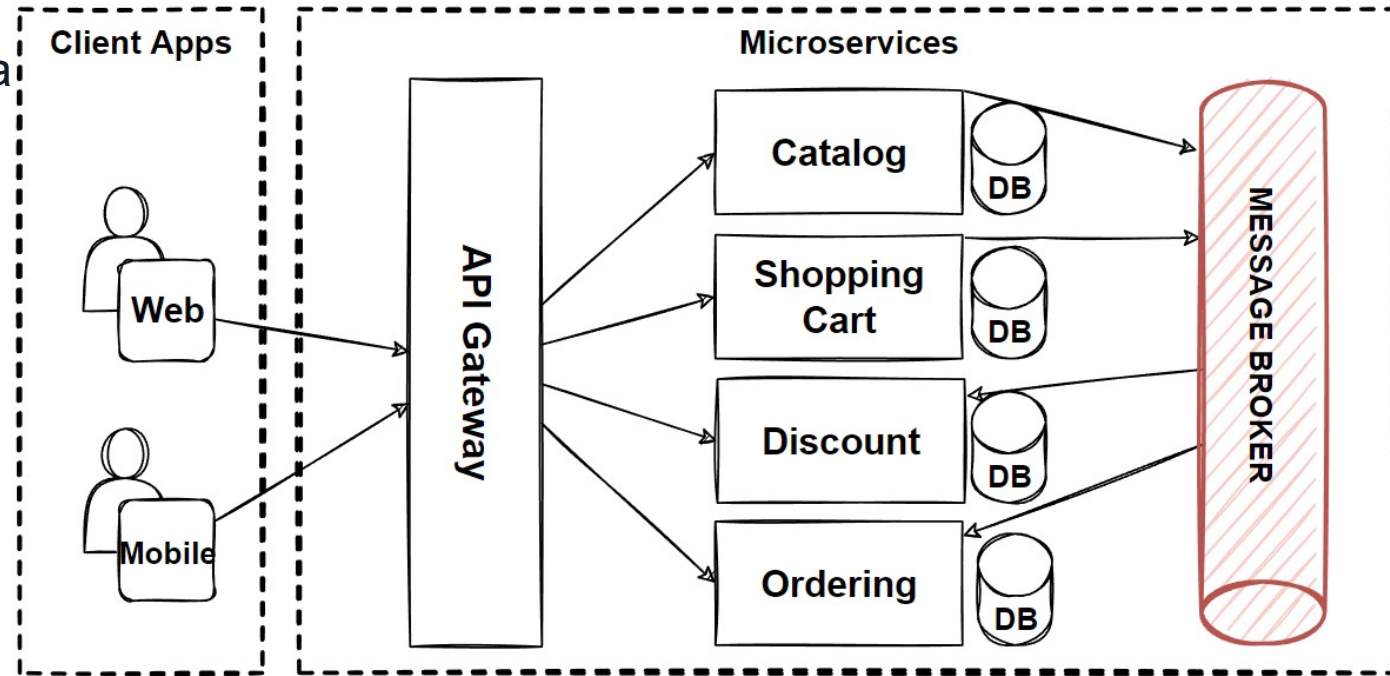


Problems

- Database are stateful service
- Scaling stateful services are not easy
- Vertical scaling has limits need to scale Horizontal
- **Different Data Requirements For Microservices**

Solutions

- Scale Stateful Application Horizontal Scaling
- Service and Data Partitioning along Business Boundaries - Shards/Pods
- Use NoSQL Database to gain partitioning
- **Identify Database Requirements following best practices**



Question

- **How to Choose a Database for Microservices ?**

How to Choose a Database for Microservices ? (Question Set)

- **Data Consistency Level**

Do we need Strict-Strong consistency or Eventual consistency ?

- Do we need ACID compliance ? Should follow Eventual consistency in microservices to gain high scalability and availability.

- **Fixed or Flexible Schema Choise, Predictable or Dynamic Data**

Are we work with fixed or flexible schema that need to change frequently, dynamically changed data ?

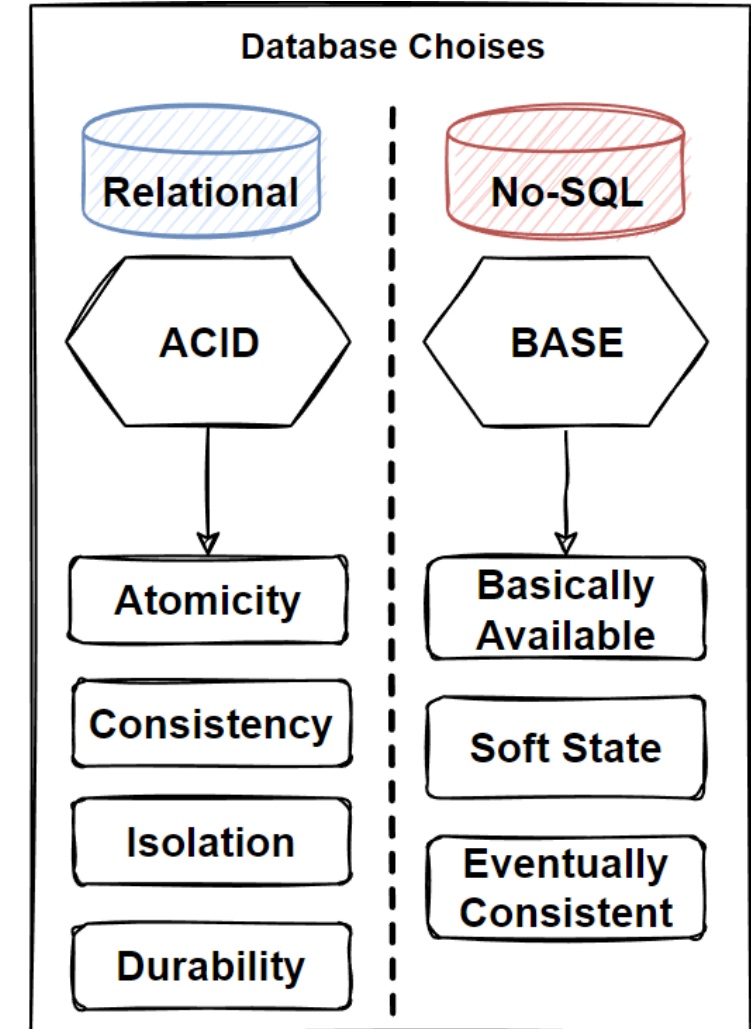
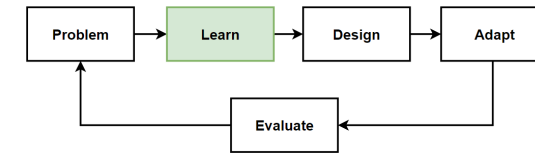
- Are we have Predictable Data or Dynamic Data ?

- **High or Low Data Volume, Predictable or Un-predictable Data**

Are we work with High Volume Data or Low Volume Data ?

- Can we have predictable data that we store our microservices database ?

- NoSQL Databases prioritize partition tolerance that handling large amount of data or data coming in high velocity.



How to Choose a Database for Microservices ? (Question Set) - 2

- **Read Requirements, Relational or non-Relational Data, Complex Join Queries**

Our data is highly structured and requires referential integrity or not required for relationships that is dynamic and frequently changes ?

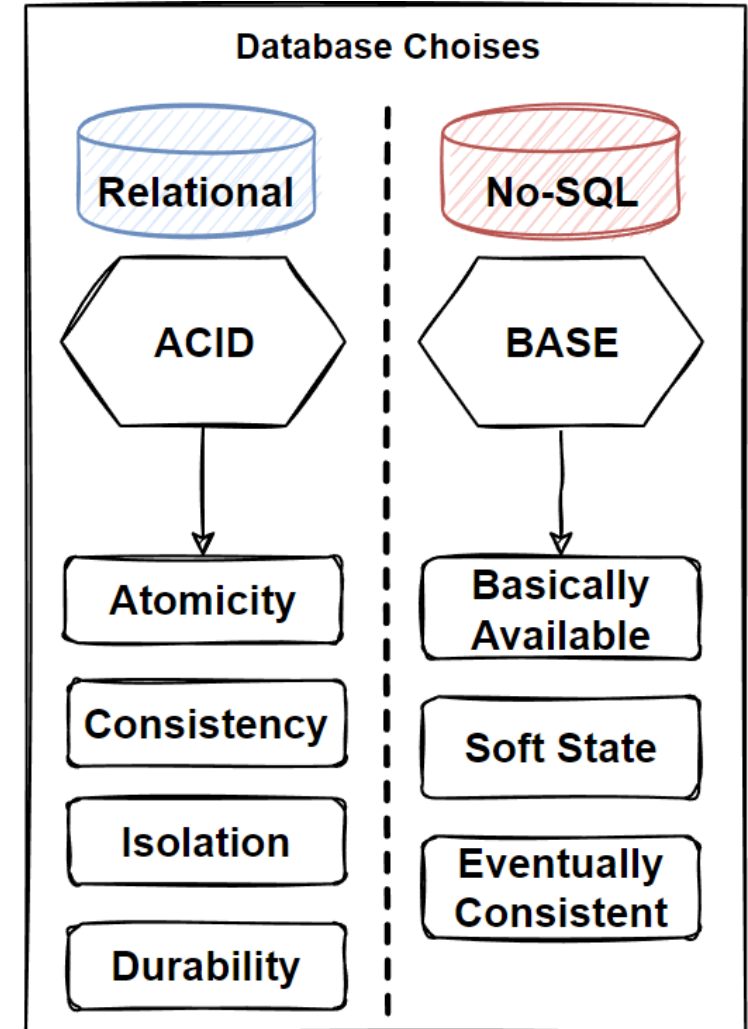
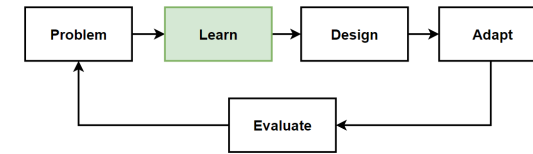
- Should it work with complex queries, table joins and run SQL queries on normalized data models or Retrieve data operations are simple and performs without table joins ?

- **Deployments, Centralized or De-centralized Structure**

Do we deployed to large and one or few locations with centralized structure ? or Do we need to deploy and replicate data across different geographical zones ?

- **High Performance Requirements**

Do we need to achieve fast read-write performance ?



How to Choose a Database for Microservices ? (Question Set) - 3

- **High Scalability Requirements**

Do we need High Scalability Requirements both vertical and horizontally scaling ?

- To accomodate millions of request should sacrifice strong consistency.

- **High Availability and Low Latency Requirements**

Do we need High Availability and Low Latency Requirements that need to separate data across different geographical zones ?

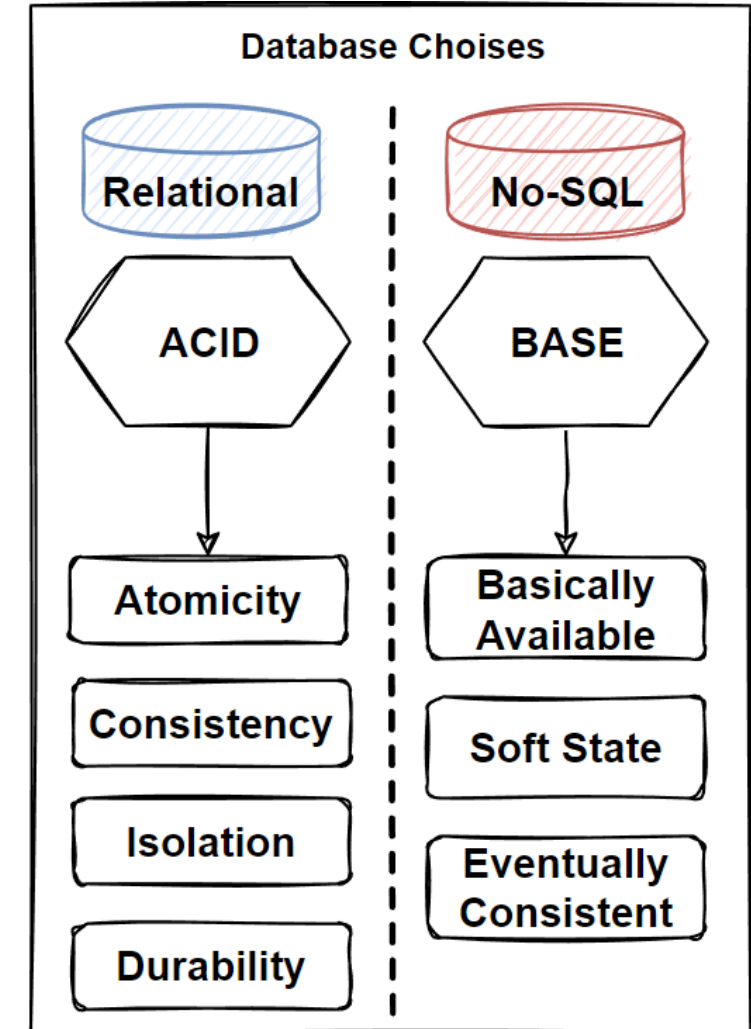
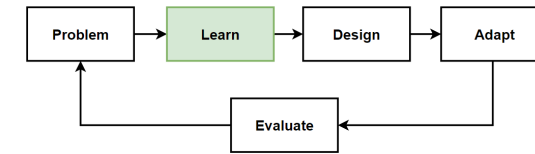
- **Can we provide ALL OF THESE FEATURES at the same time ?**

Is it possible to provide High Scalability, High Availability and Low Latency with High Performance and able to run Complex Join Queries providing with ACID principles strong data consistency ?

- **Unfortunately, NO.**

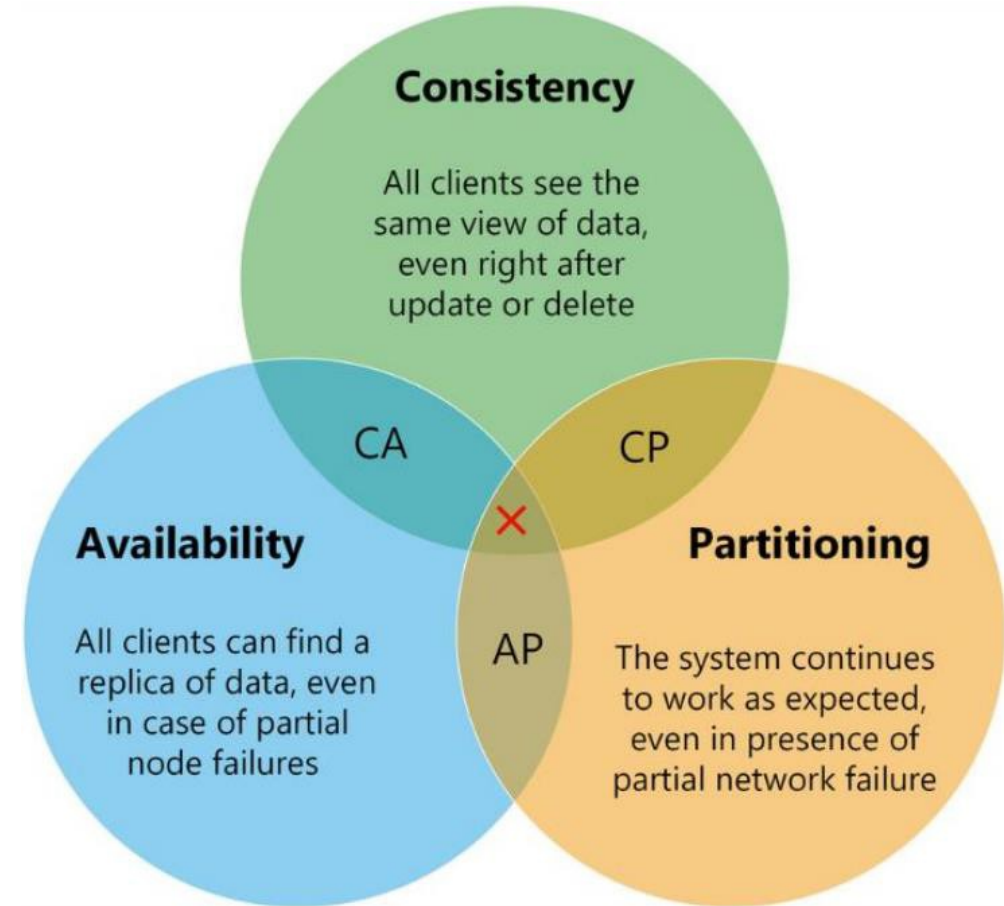
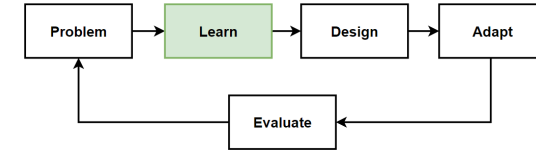
- **WHY ?**

- **The CAP Theorem.**



CAP Theorem

- Found in 1998 by a **professor Eric Brewer**.
- **Consistency**, **Availability**, and **Partition Tolerance** cannot all be achieved **at the same time**.
- Distributed systems **should sacrifice** between **consistency**, **availability**, and **partition tolerance**.
- Any database can **only guarantee two of the three concepts**; **Consistency**, **Availability**, and **Partition tolerance**.



CAP Theorem – Consistency, Availability, Partition Tolerance

- **Consistency**

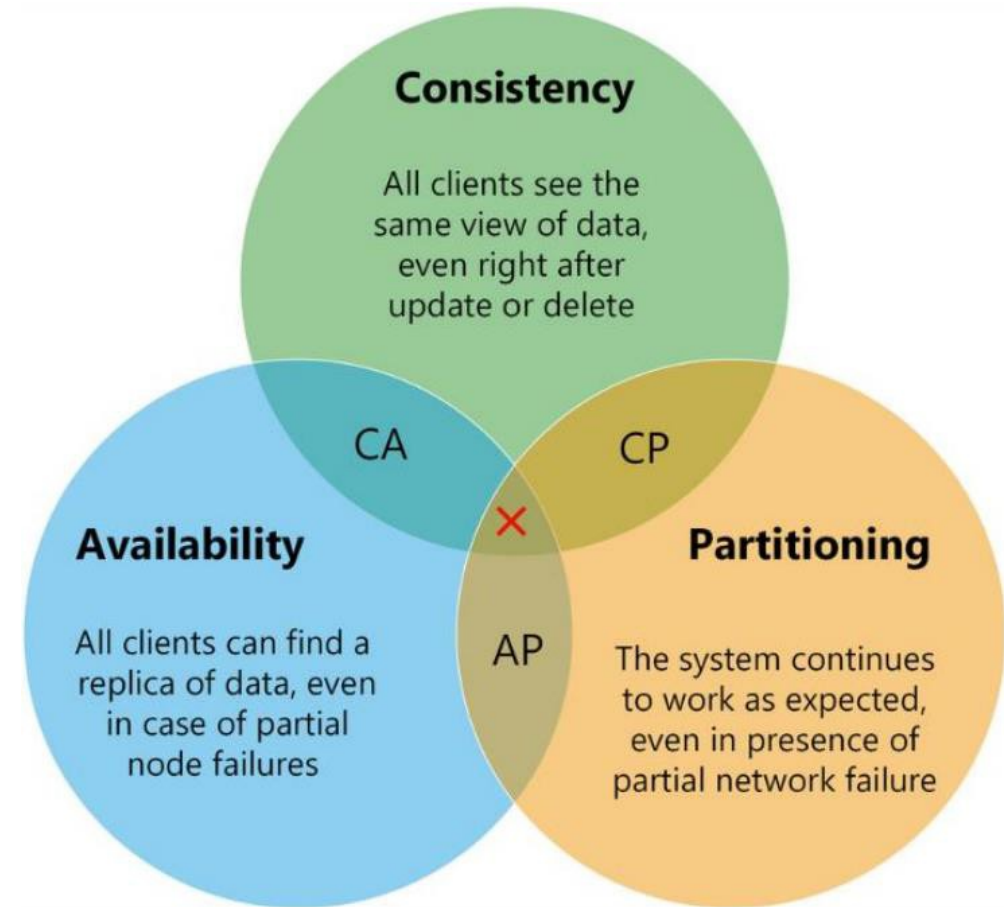
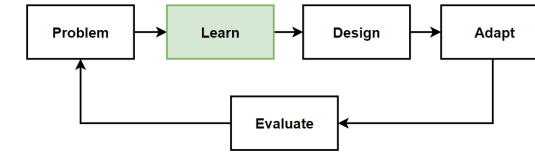
When the system get any read request, the data should return last updated value from database under all circumstances. If the data cannot be retrieved, an error should be throw. When consistent not provide, the system must block the request until all replicas update.

- **Availability**

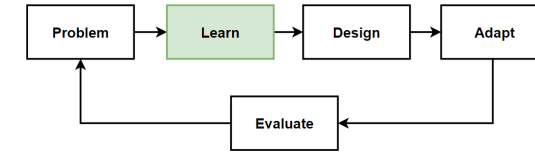
The ability of a distributed system to respond to requests at any time. That is a distributed system can respond all request any time. Even if one node in any cluster is down, the system should be able to survive with other nodes. (fault-tolerance)

- **Partition Tolerance**

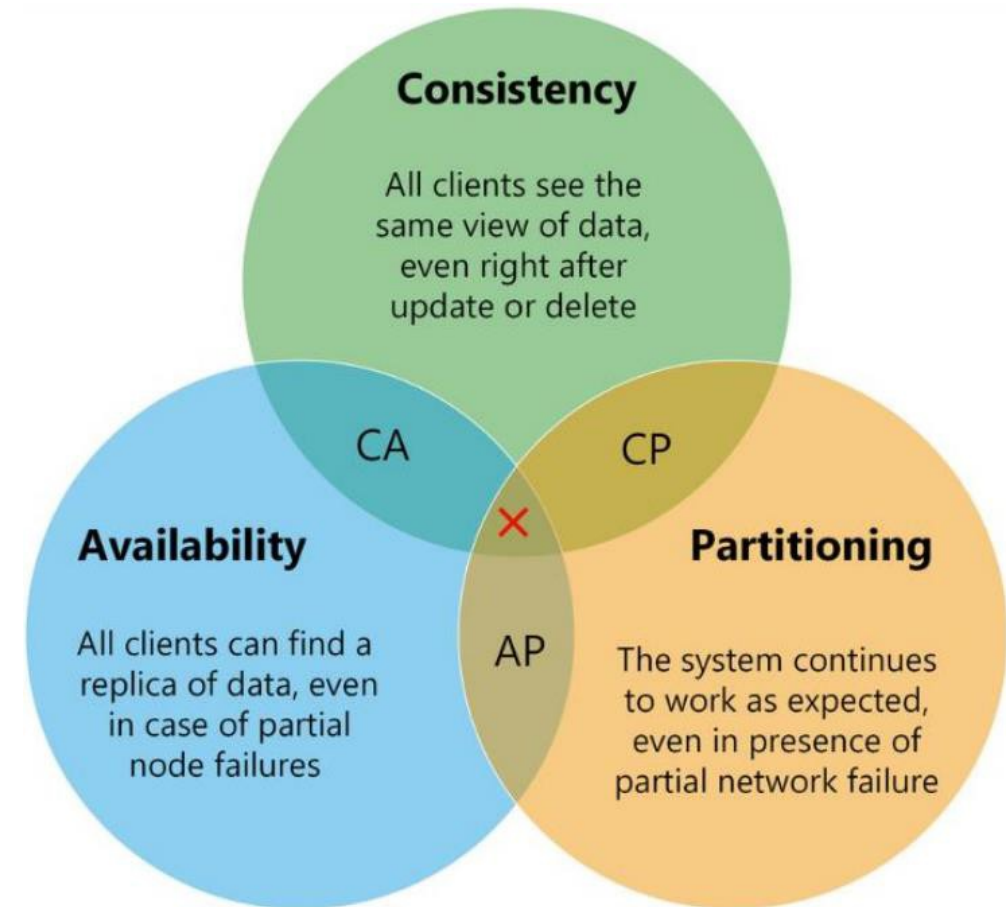
Network partitioning; parts of system are located in different networks. The ability of the system to continue its life in case of any communication problem. Guarantees the system continues to operate if one node is down.



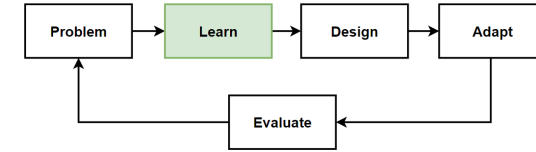
Consistency and Availability at the same time ?



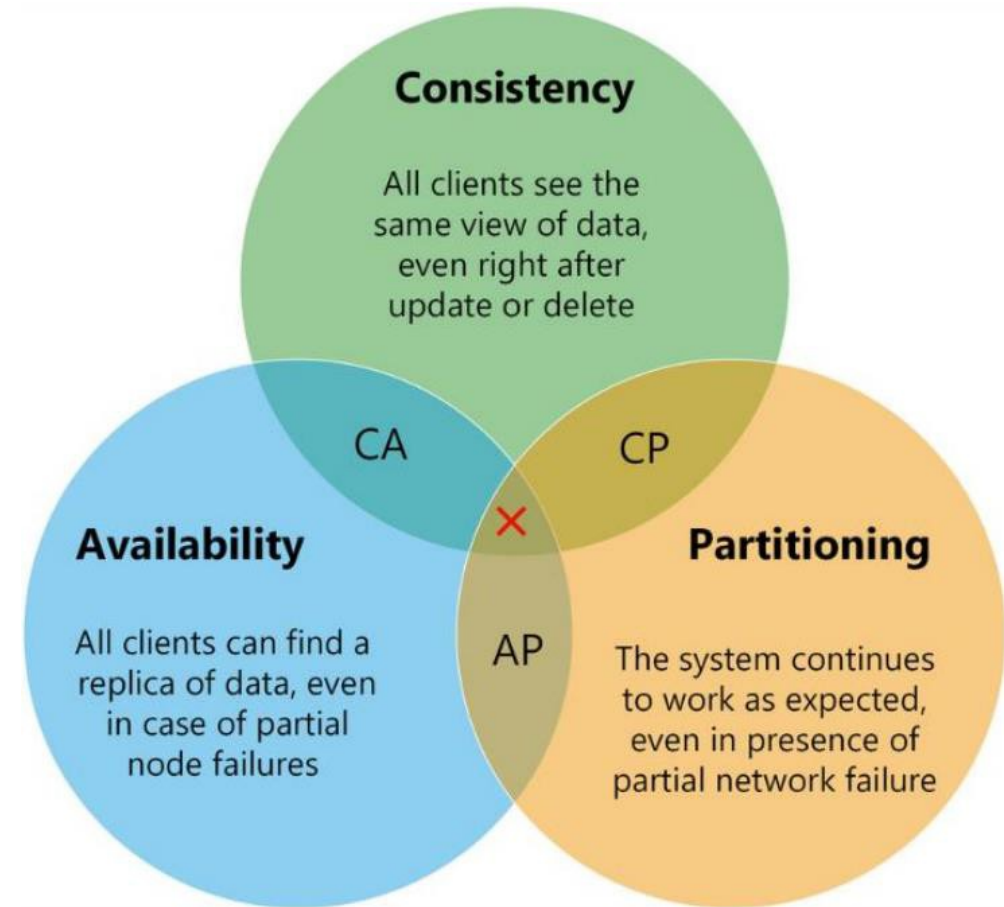
- If there is **Partition Tolerance**, either Availability or Consistency should be selected. **Should sacrifice Availability or Consistency** in distributed systems.
- **Partition Tolerance is a must** for distributed architectures. The emergence of NoSQL databases is to easily **overcome** the **Single Point of Failure** problem.
- Relational databases **prevent distribute data** from different nodes. **NoSQL** databases **don't include foreign keys, joins**, that is, relations between data.
- **Un-related data** allows it to be stored in a **distributed manner**, NoSQL databases easily scalable.
- Distributed system **doesn't have the luxury of not providing Partition Tolerance** anyway.
- **MongoDB, Cassandra**; None of them gave up on Partition Tolerance, and made a choice between Availability and Consistency.



Consistency and Availability at the same time ? - 2

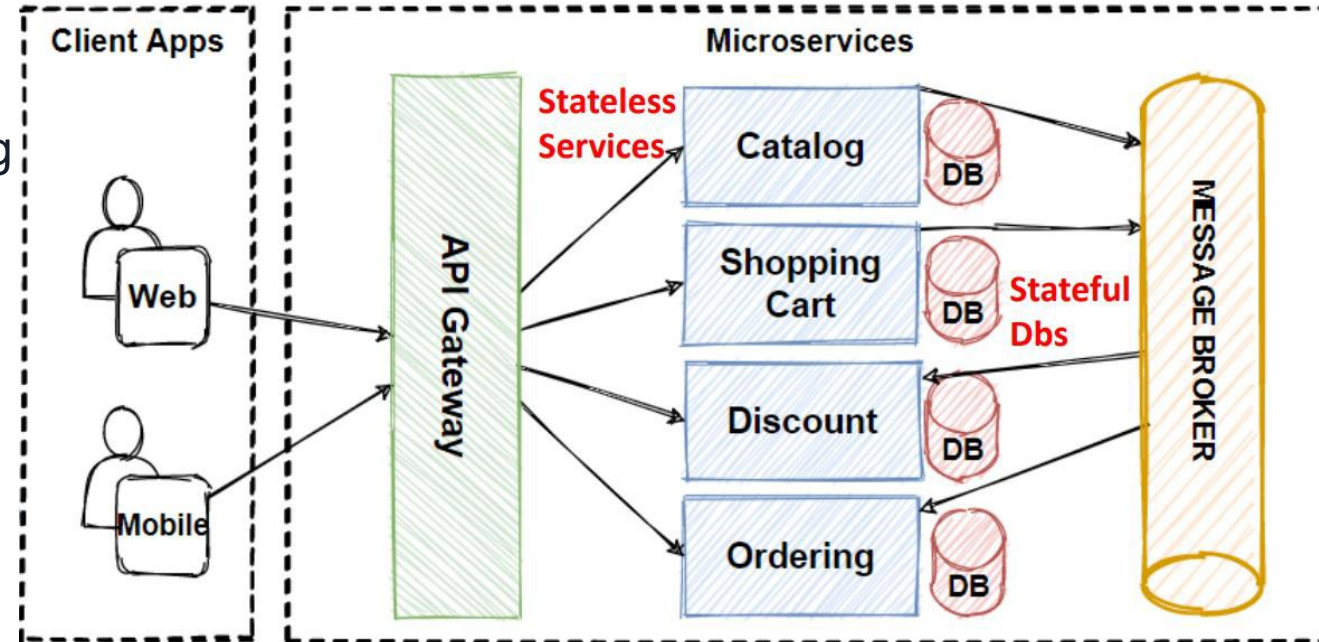
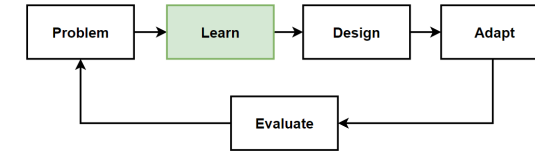


- In Distributed architecture, **Partition Tolerance** seems to be a **must-have feature**. it is necessary to choose between Consistency or Availability.
- If a system is to be **fully consistent**, it **must be sacrifice** that always available. (**High Availability**)
- If it is **desired** to be **accessible** at **all times**, the **consistency** should be **sacrificed**.
- In **microservices architectures** choose **Partition Tolerance** with **High Availability** and follow **Eventual Consistency**.



Scale Database in Microservices

- **Problem of Single Database Server**
- Why we need to use **Data Partitioning** ?
- **Scaling databases** in microservices architecture
- **Splitting database** servers with Database Sharding
- Single Database Server leads to **low performance**



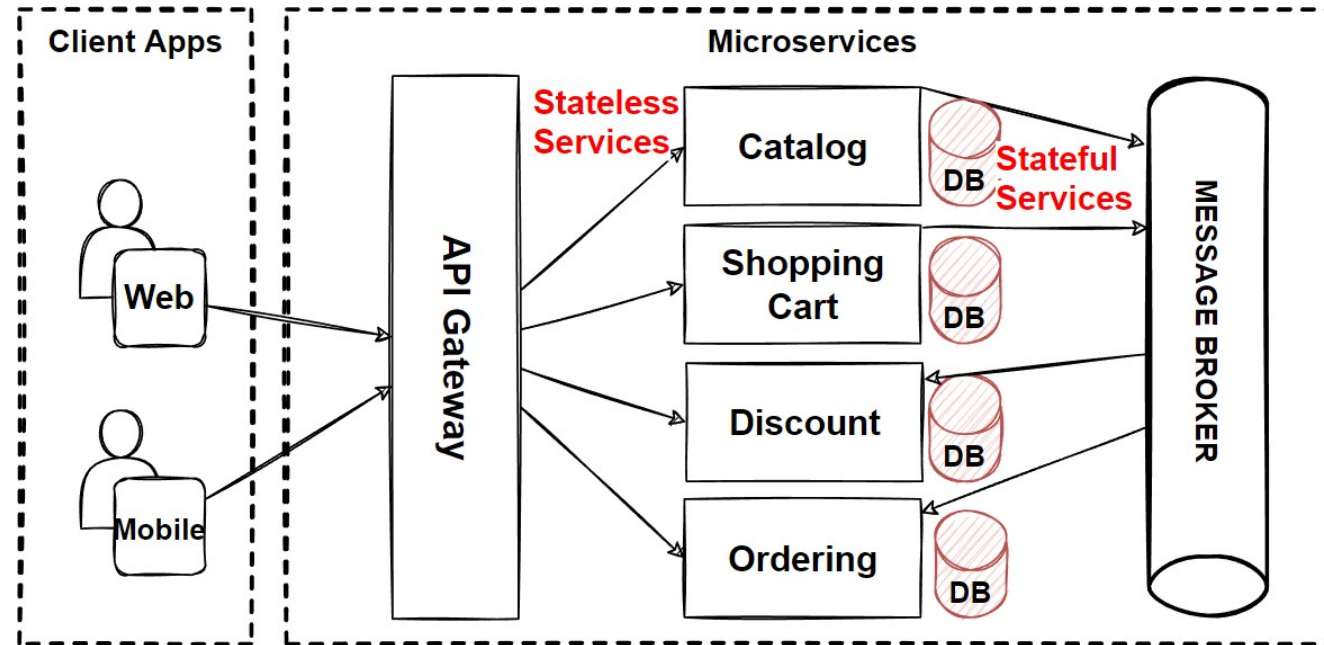
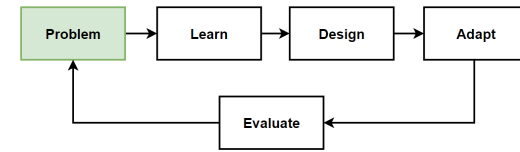
Problem: Single Database Server Performs Low Performance

Problems

- Databases become very slow by time with app grows
- Single database goes un-manageable situation
- Database operations are become slower
- Very low performance due to slowness of single database
- Running out of disk space

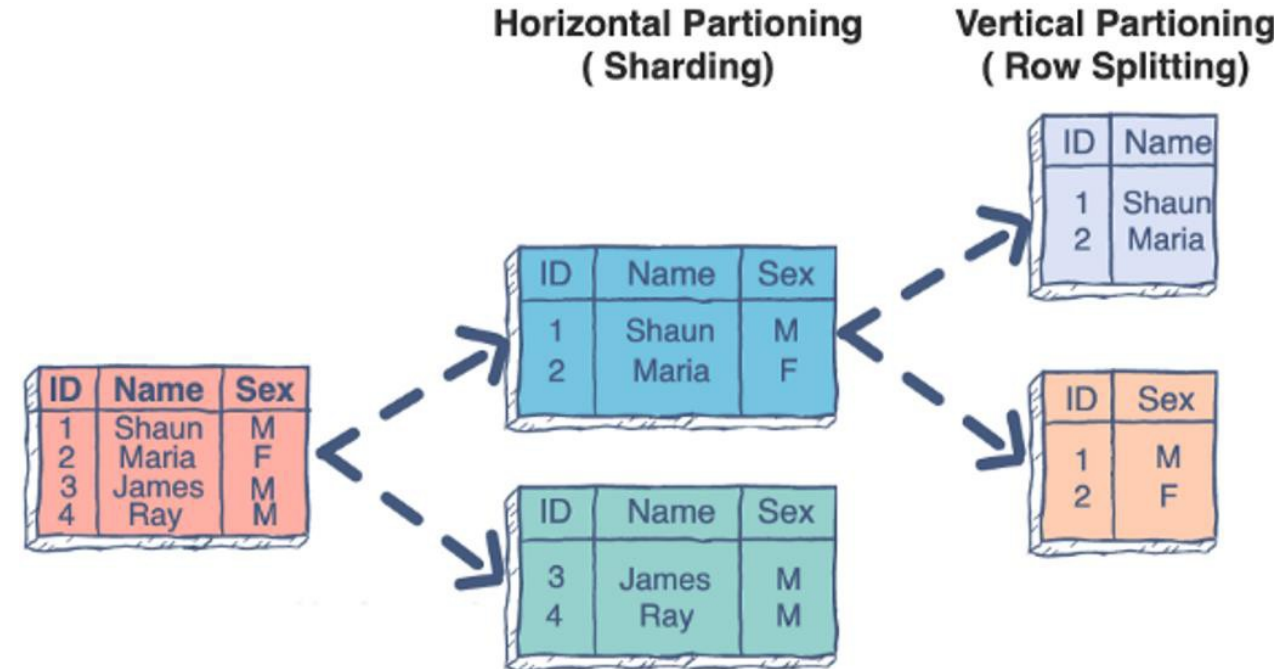
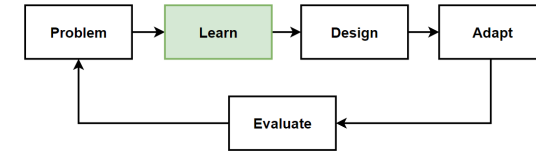
Solutions

- Data Partitioning
- Horizontal, Vertical, and Functional Data Partitioning
 - Improve availability
 - Increase scalability
 - Increase performance
 - Improve security
 - Improve data management, Operation

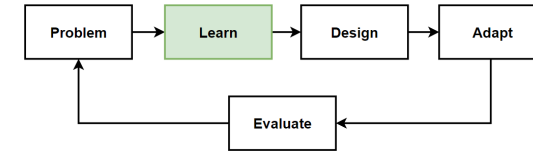


What is Data Partitioning ?

- **Data Partitioning** is divides a large dataset into several small partitions.
- Data is **divided into partitions** that can be easily managed and accessed separately.
- **Distributes data across different partitions** to improve database availability, scalability and performance.
- **Abstracting partitioning** architecture from the client applications.
- **Client applications** seems as a **single unit** databases.



Why we are using Data Partitioning ?



- **Increase Scalability**

When divide data across multiple data partitions into separate db servers, we can scale out the system without worrying any hardware limits.

- **Improve Availability**

With Separating data across multiple servers, we protect our systems for a single point of failures.

- **Increase Performance (Query and operations)**

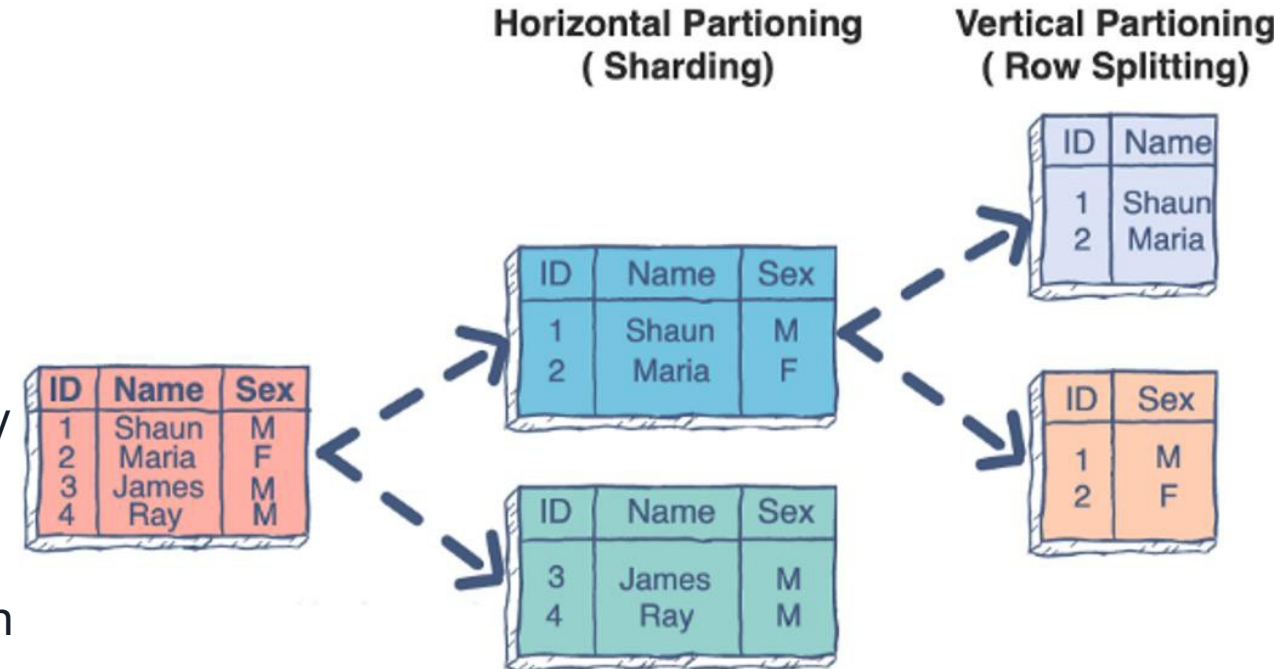
Instead of querying whole database, system query only smaller components.

- **Improve Security**

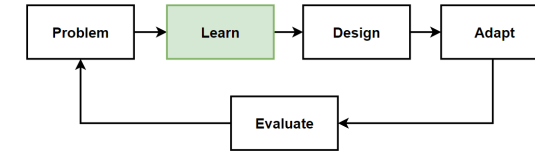
By storing sensitive and non-sensitive data into different partitions.

- **Improve Data Management**

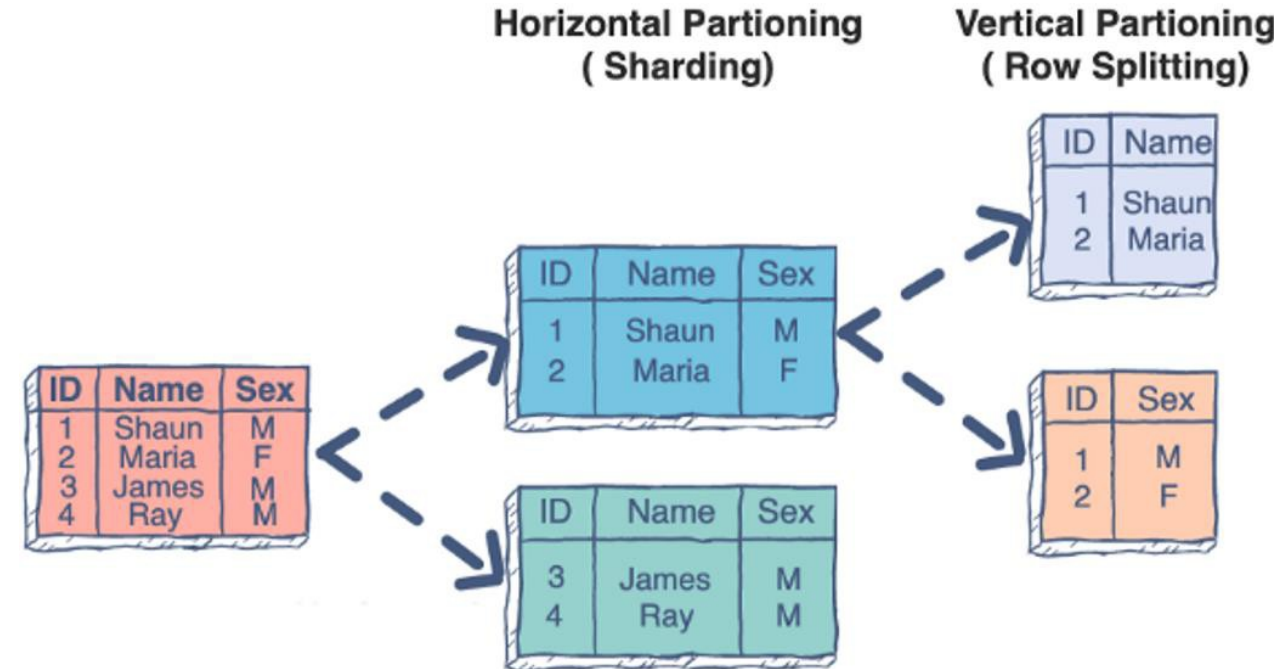
Due to divide tables and indexes with different partitions, its more easy to manage small units and easy to maintenance.



Horizontal, Vertical, and Functional Data Partitioning

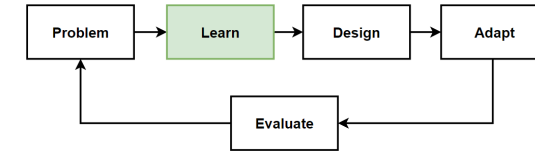


- There are three methods for partitioning data:
- Horizontal Partitioning (often called sharding) and we mostly use this partitioning
- Vertical Partitioning
- Functional Partitioning



Horizontal Partitioning - Sharding

- **Horizontal partitioning** is also called **sharding**.
- **Each partition** is a **separate data store**, but all partitions have the same schema.
- Shards holds a **specific subset of the data**.
- **Splitting the table data** horizontally based on **partition key**.
- Tables are **divided to smaller tables** that table rows are stored different partitions.
- **The partition key** provide to distribute data among all the **partitions**.
- With the partition key, database **locate the specific partition**.
- Data requests are **spreading the load** over more servers and provide more performance.
- The **disadvantages** of **horizontally partitioning** could be the setting **wrong partition key**.



Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver

Vertical Shards

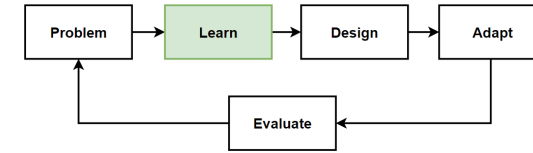
VS1			VS2	
CUSTOMER ID	FIRST NAME	LAST NAME	CUSTOMER ID	CITY
1	Alice	Anderson	1	Austin
2	Bob	Best	2	Boston
3	Carrie	Conway	3	Chicago
4	David	Doe	4	Denver

Horizontal Shards

HS1				HS2			
CUSTOMER ID	FIRST NAME	LAST NAME	CITY	CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin	3	Carrie	Conway	Chicago
2	Bob	Best	Boston	4	David	Doe	Denver

[An Overview of Sharding & Partitioning | Hazelcast](#)

Vertical Partitioning



- **Vertical partitioning** is a "Row Splitting".
- **Each partition** holds a **subset of the columns** for table in the database.
- The data **divided based on columns** and we can divide by **mostly visited columns** and the other columns in different servers.
- **Frequently accessed columns** can be use in one vertical partition and less frequently accessed fields in another.
- **Example case:** Facebook, the user profile data can be in different partition and user mostly visited data can be different server.
- **Benefit of Vertical Partitioning** is you can separate the critical and mostly visited columns in a separate server.
- **Best practice:** Divide by rarely-changes and frequently-change columns into different servers.

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver

Vertical Shards

VS1			VS2	
CUSTOMER ID	FIRST NAME	LAST NAME	CUSTOMER ID	CITY
1	Alice	Anderson	1	Austin
2	Bob	Best	2	Boston
3	Carrie	Conway	3	Chicago
4	David	Doe	4	Denver

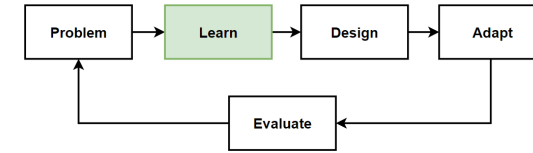
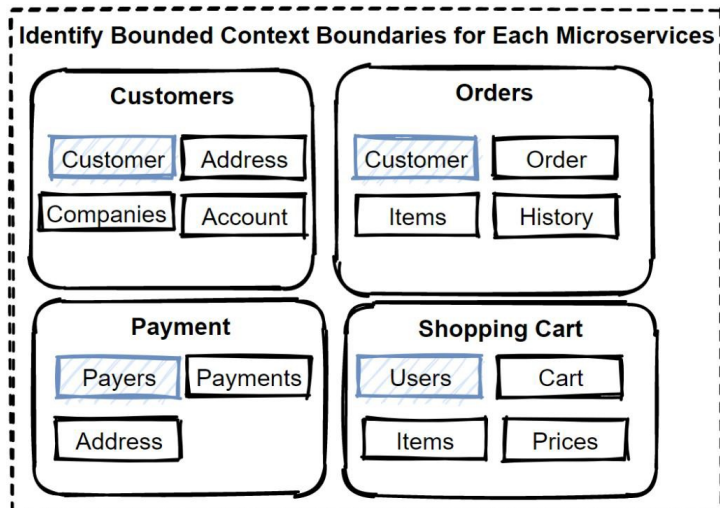
Horizontal Shards

HS1			
CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin
2	Bob	Best	Boston
HS2			
CUSTOMER ID	FIRST NAME	LAST NAME	CITY
3	Carrie	Conway	Chicago
4	David	Doe	Denver

[An Overview of Sharding & Partitioning | Hazelcast](#)

Functional Partitioning

- The **data is divided** by following the **bounded context** or **subdomains**.
- **Data is segregated** according to **usage of bounded contexts** in the system.
- Its like **decomposing microservices** as per responsibilities with considering bounded contexts.
- **Functional partitioning** into **Microservices Decomposition** section.
- Apply functional partitioning following the **Decomposition patterns**:



3 Dimensions of Scaling

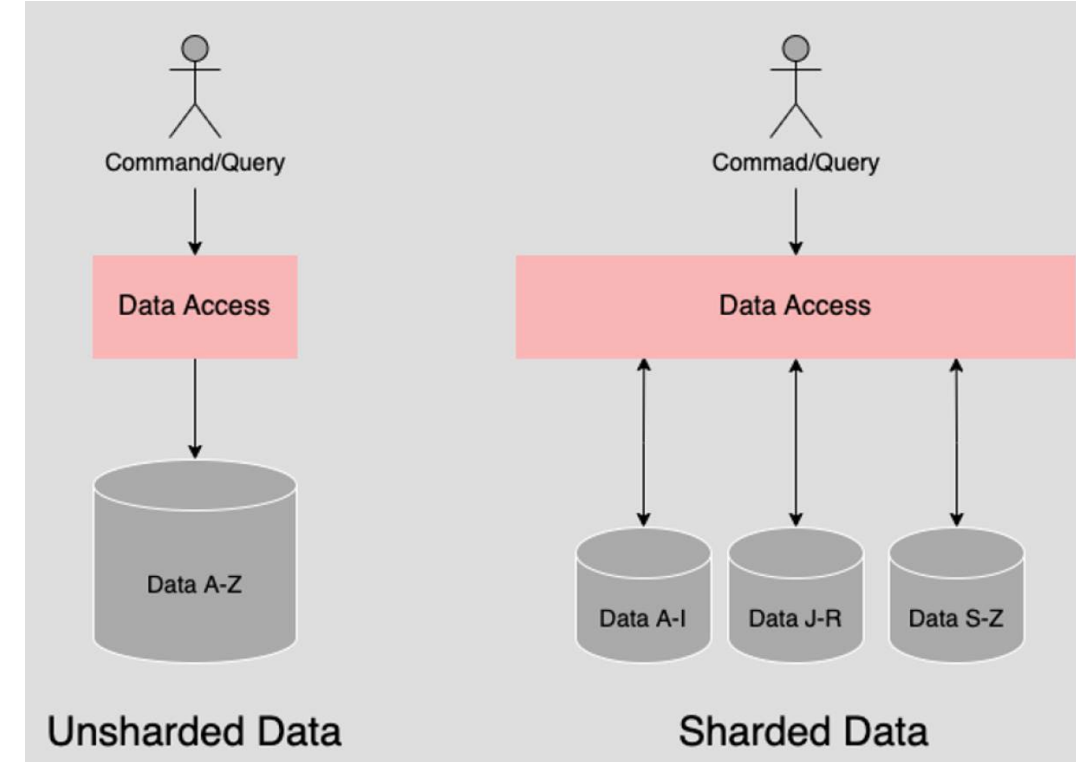
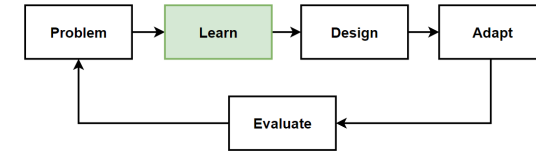
Y axis - functional decomposition
Scale by splitting small services

X axis - horizontal duplication
Scale by Clonning

Z axis - data partioning
Scale by splitting data

Database Sharding Pattern

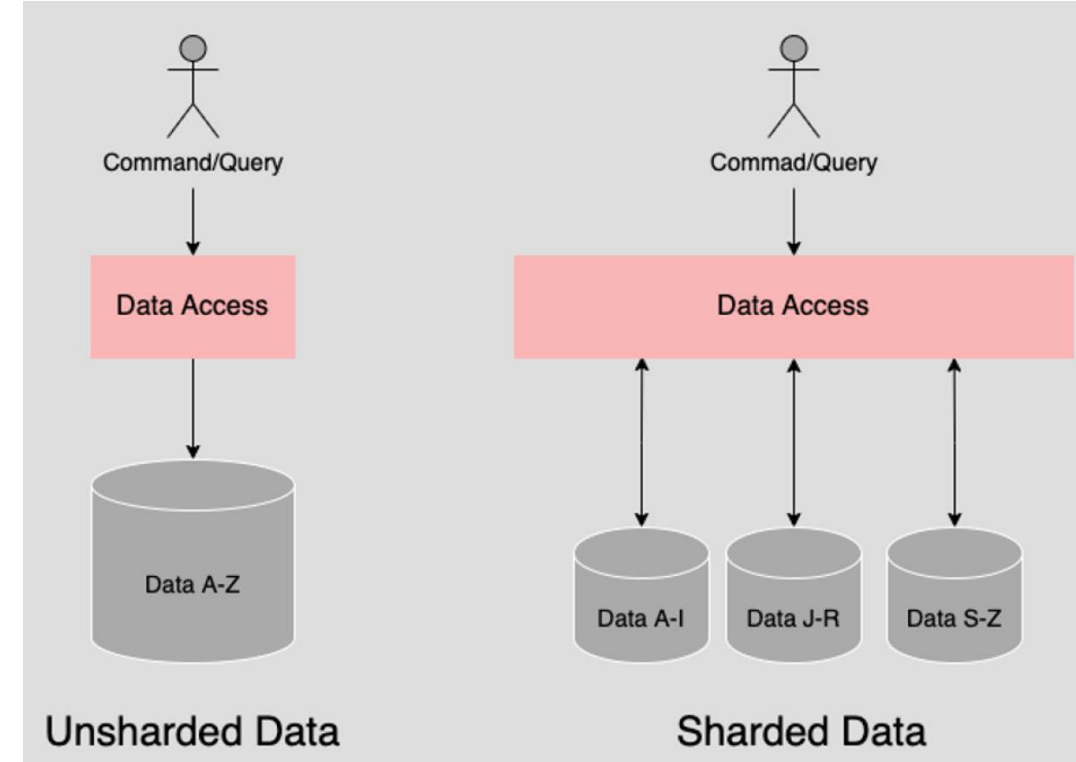
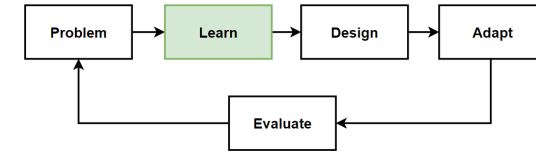
- **Sharding** is "a small piece or part", meaning to be a small piece or part of something.
- **Database sharding** is the **separation of the data** into unique small pieces of database which called shards or chunks.
- When storing and accessing large volumes of data in microservices architectures, we should **divide a databases** into a **set of horizontal partitions** or **shards**.
- **Each shard** has the **same schema**, holds its own distinct subset of the data.
- Shardings **enable to scale** the system with adding new shards according to storage needs.
- It can **improve performance** and **reduce contention** by balancing the workload across different shards.
- We can **gain cloud power** with locating physical servers closely which users can access the data performantly.



<https://www.redhat.com/architect/pros-and-cons-sharding>

Database Sharding Pattern - 2

- **Databases** dividing into **shards** with **partition keys**.
- These **partition keys** provide to decide which data should be placed in each shard.
- The **partition key** should be **static**. It shouldn't affect by the data changes.
- **Database Sharding** provides dividing a **data store** **physically** and **organizes** the data into **shards**.
- When an application **retrieves data** by **querying database**, the sharding logic **redirects** the application to **the correct shard server**.
- All these database sharding operations are **abstracting the physical location** of the data in the sharding logic from client applications.



<https://www.redhat.com/architect/pros-and-cons-sharding>

Benefits of Database Sharding Pattern

- **Increase Scalability**

When divide data across multiple data partitions into separate db servers, we can scale out the system without worrying any hardware limits.

- **Improve Availability**

With Separating data across multiple servers, we protect our systems for a single point of failures.

- **Increase Performance (Query and operations)**

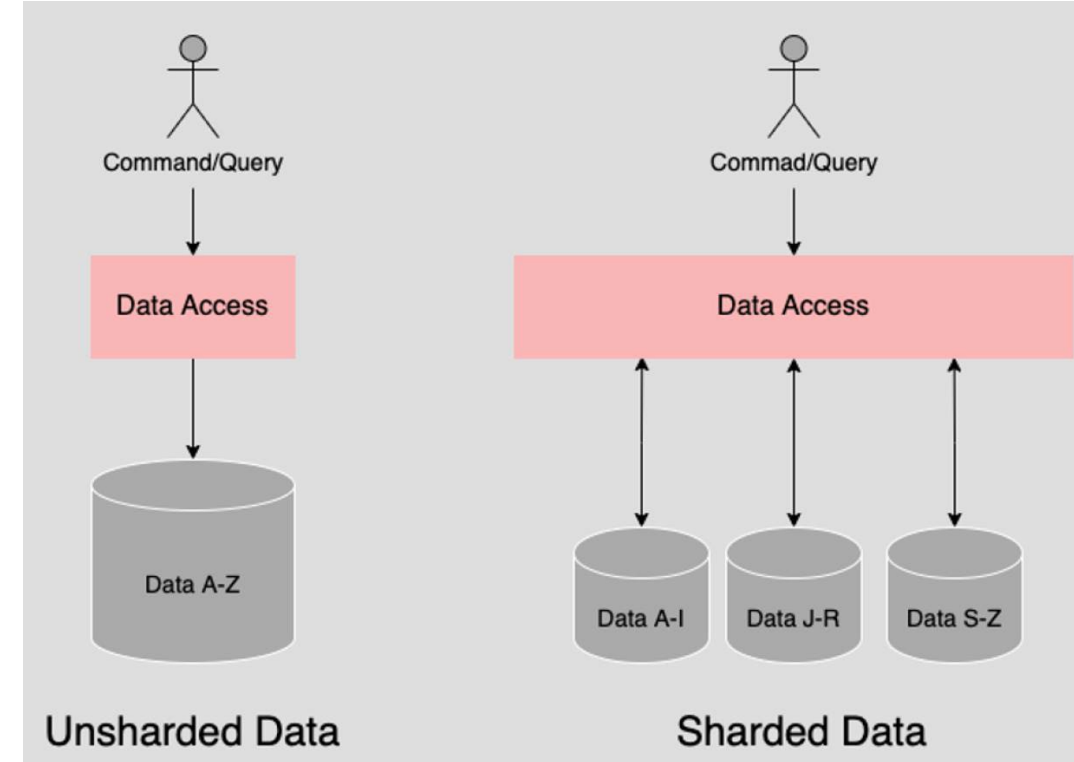
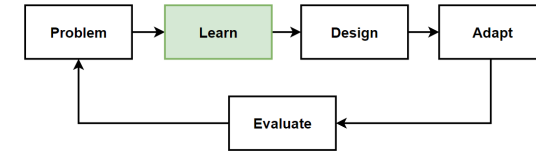
Instead of querying whole database, system query only smaller components.

- **Improve Security**

By storing sensitive and non-sensitive data into different partitions.

- **Improve Data Management**

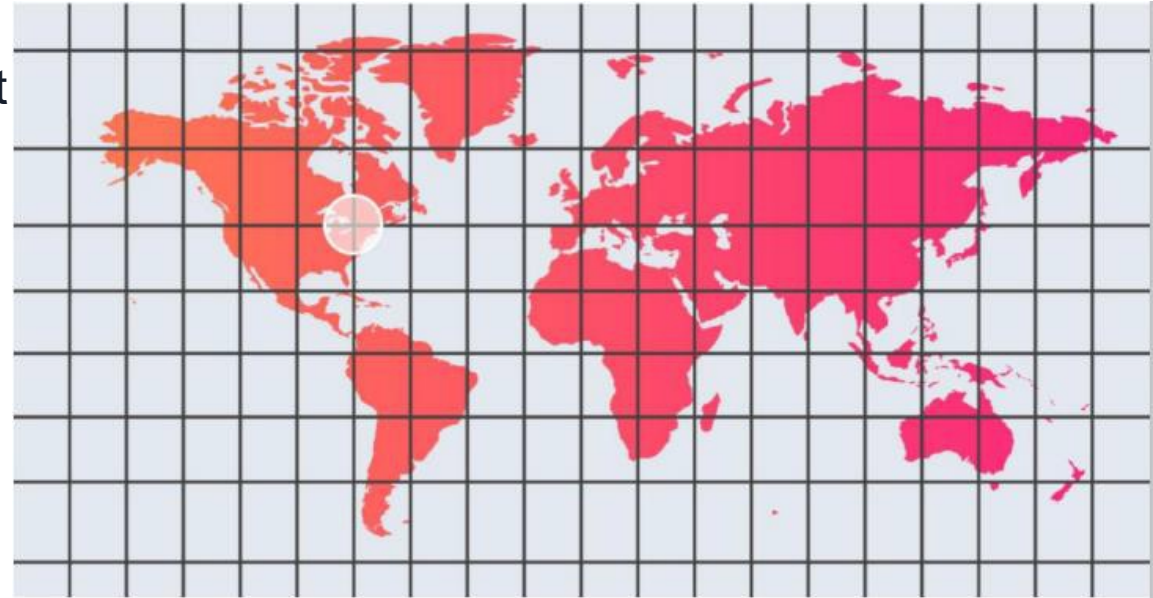
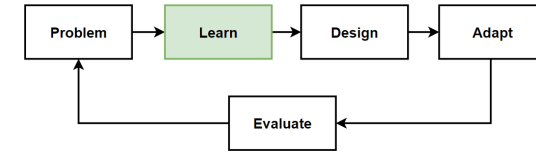
Due to divide tables and indexes with different partitions, its more easy to manage small units and easy to maintenance.



<https://www.redhat.com/architect/pros-and-cons-sharding>

Tinder System Design Example of Database Sharding Pattern

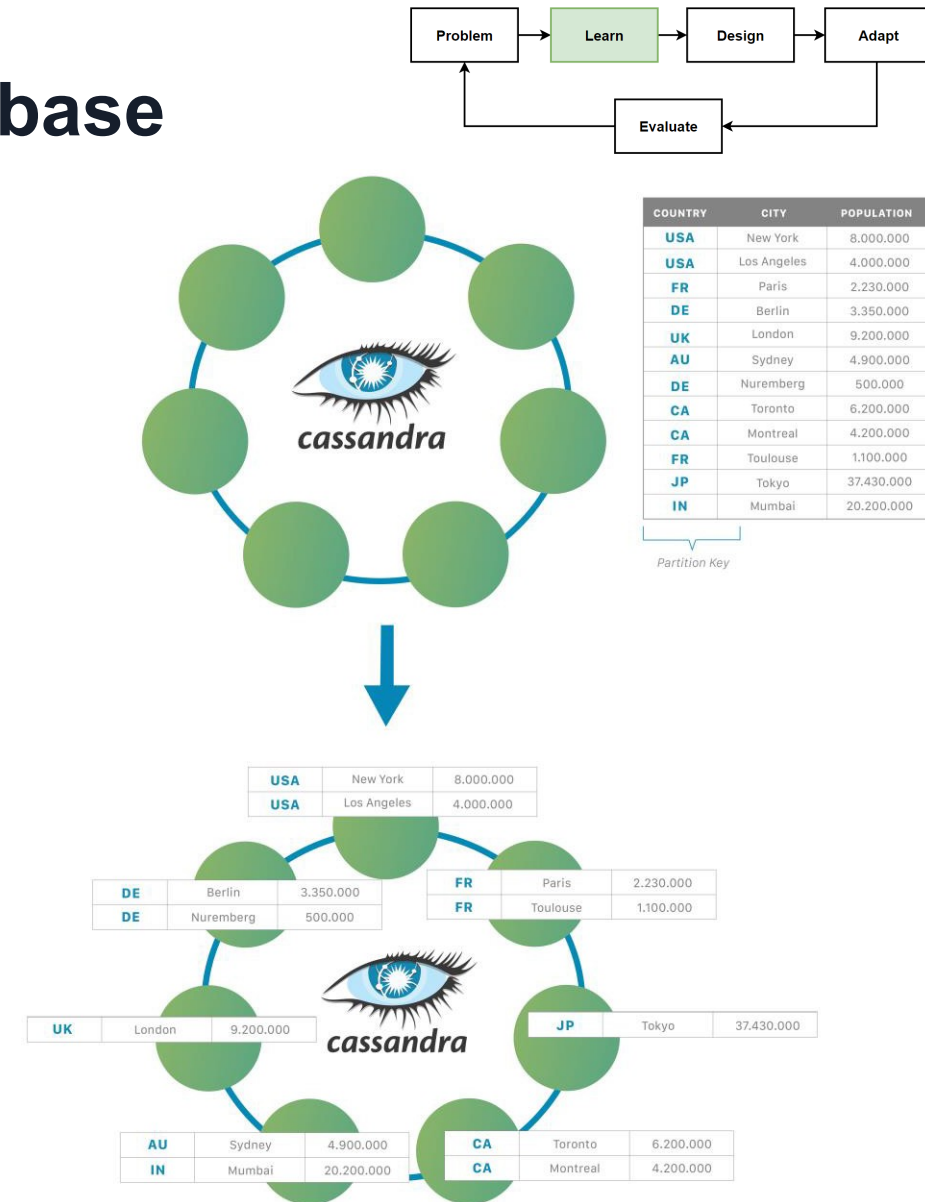
- **Tinder** is very good example of Database **Sharding Pattern**.
- **Tinder** allows to **match** and **meet** other **people** who use the application near you (around 160km) **based on location**.
- To **find people near you very quickly** and offer choices that meet the criteria what you set.
- How **Tinder match peoples** who are **near to each other** ?
- Tinder **segments users** based on **their location**.
- This is called **GeoSharding**, that is, **location-based database sharding**.
- Sharding **by dividing the world map into boxes** with their locations and matches them in only into that box locations in the world.



Cassandra No-Sql Database

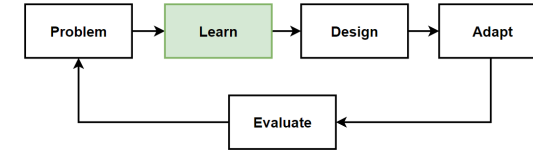
Peer-to-Peer Distributed Wide Column Database

- **Cassandra** is very good example of implementation **Database Sharding Pattern** and using **Horizontal Scaling** features.
- **Cassandra** is a **distributed database** from Apache Foundation.
- It is **highly scalable** and designed to manage very **large amounts of structured data**.
- It provides **high availability** with **no single point of failure**.
- **Apache Cassandra** is a highly scalable, high-performance **distributed database** designed to **handle large amounts** of data across many **different located servers**.
- **Apache Cassandra features;**
 - Elastic scalability - It is scalable, fault-tolerant, and consistent.
 - No single point of failure.
 - Column-oriented database.
 - Flexible data storage.
 - Easy data distribution.

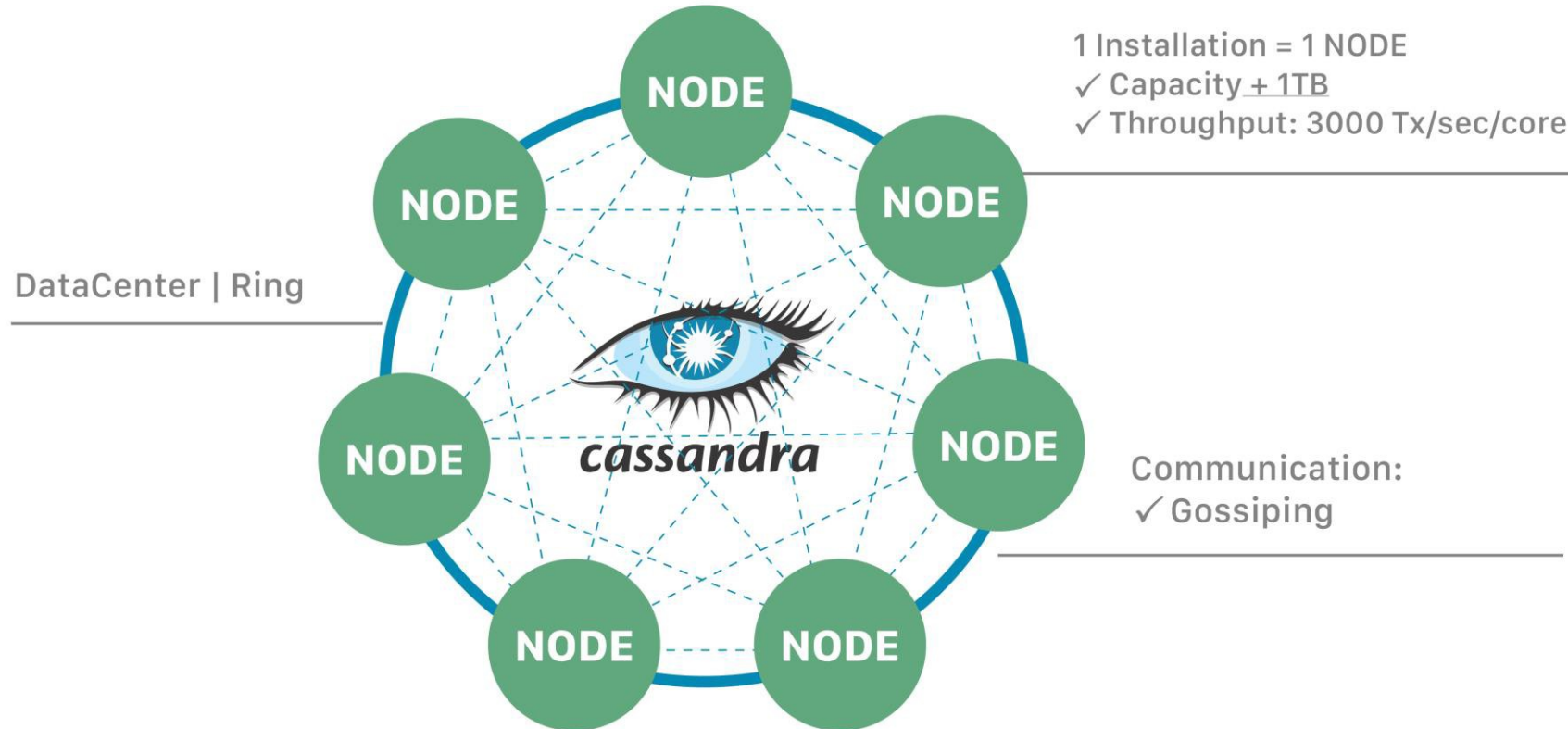


https://cassandra.apache.org/_/cassandra-basics.html

Cassandra Architecture



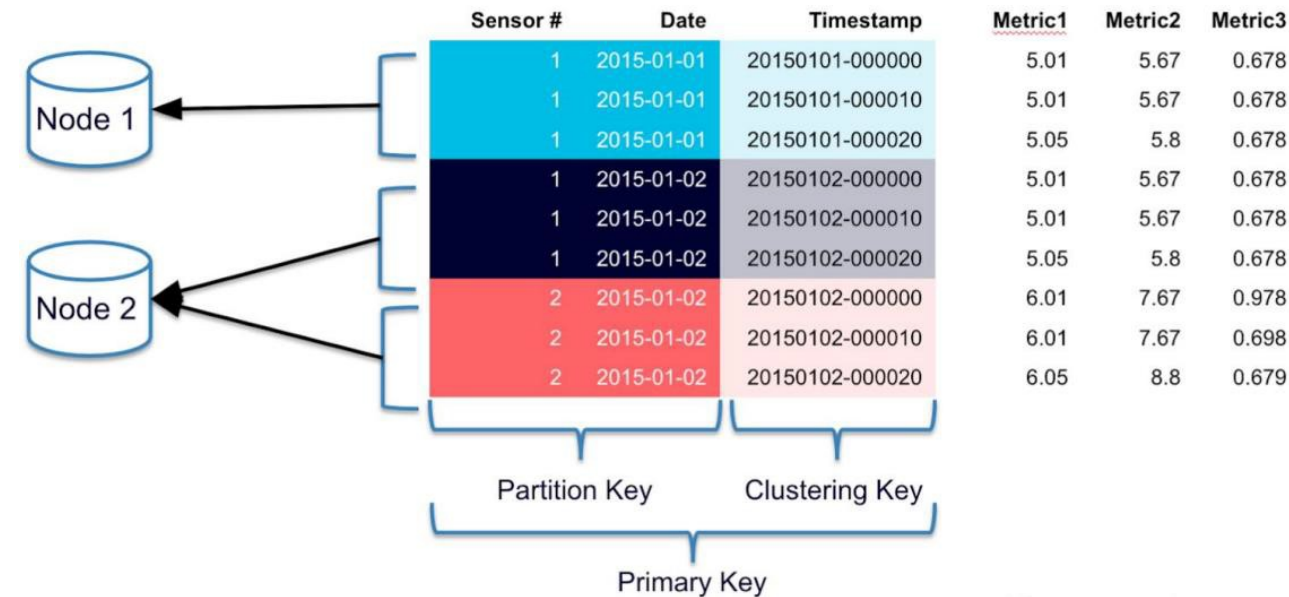
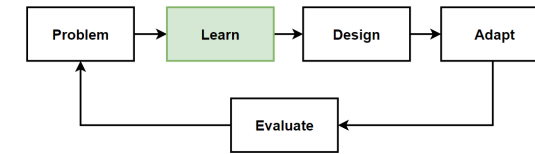
ApacheCassandra™ = NoSQL Distributed Database



https://cassandra.apache.org/_/cassandra-basics.html

Why Cassandra No-Sql Database ?

- **Cassandra** has an **auto-sharding** feature and it helps keep data divided among nodes.
- All of the columns that will **decide on which node** the data should be kept are called "**Partition Key**".
- **Partition Key** is set to **Sensor#** and **Date**.
- The data will be **stored in nodes** determined by inserting the value of these **two columns** with hash function. It becomes **Partition Keys**.
- **CAP Theorem**: either Availability or Consistency must be selected in a distributed system.
- **Microservice architectures** decided that **Availability is more important** to them and sacrifice for strict consistent and **embrace to Eventual Consistency** would be sufficient.



https://cassandra.apache.org/_/cassandra-basics.html

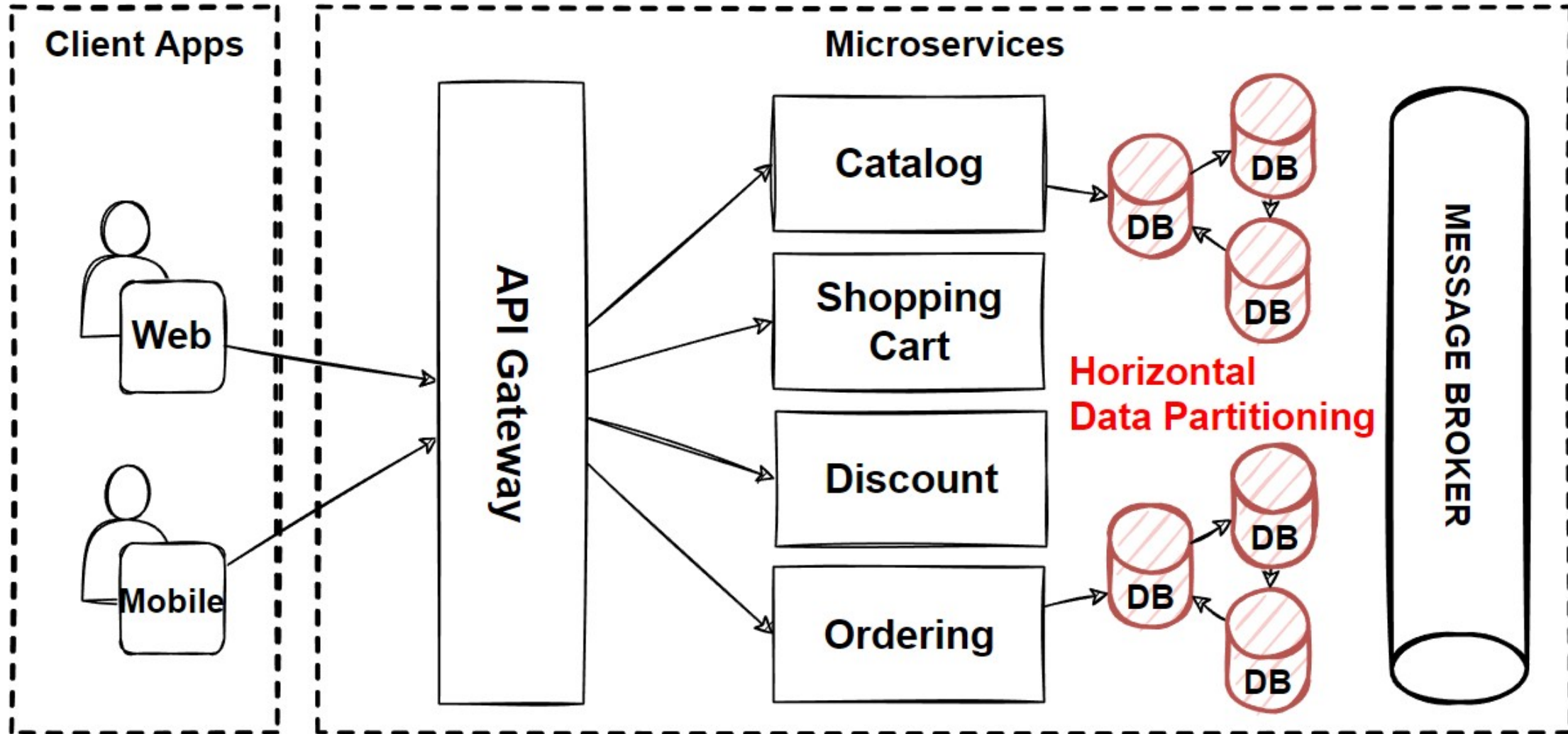
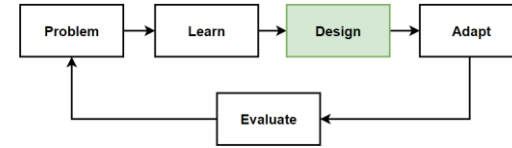
Before Design – What we have in our design toolbox ? - Previous

Architectures	Patterns&Principles	Microservices Communications	Microservices Async Communications	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns	• HTTP Based RESTful API • GraphQL API • gRPC API • WebSocket API • Gateway Routing Pattern • Gateway Aggregation Pattern • Gateway Offloading Pattern • API Gateway Pattern • Backends for Frontends Pattern-BFF • Service Aggregator Pattern. • Service Registry/Discovery Pattern	• Single-receiver Message-based Communication (one-to-one model) • Multiple-receiver Message-based Communication (one-to-many model-topic) • Dependency Inversion Principles (DIP) • Fan-Out Publish/Subscribe Messaging Pattern • Topic-Queue Chaining & Load Balancing Pattern	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history Non-FR • High Scalability • High Availability • Millions of Concurrent User • Independent

Before Design – What we have in our design toolbox ? - Data

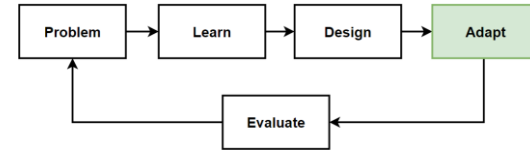
Architectures	Patterns&Principles	Microservices Data Management	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns	• The Shared Database Anti-pattern • Relational and NoSQL Databases • CAP Theorem – Consistency, Availability, Partition Tolerance • Data Partitioning: Horizontal, Vertical and Functional Data Partitioning • Database Sharding Pattern	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with Database Sharding Pattern



Peer-to-Peer Distributed Wide Column Database
Master-Master (Master-less) architecture

Adapt: Microservice Architecture with Database Sharding Pattern - Cassandra



Frontend SPAs

- Angular
- Vue
- React

API Gateways

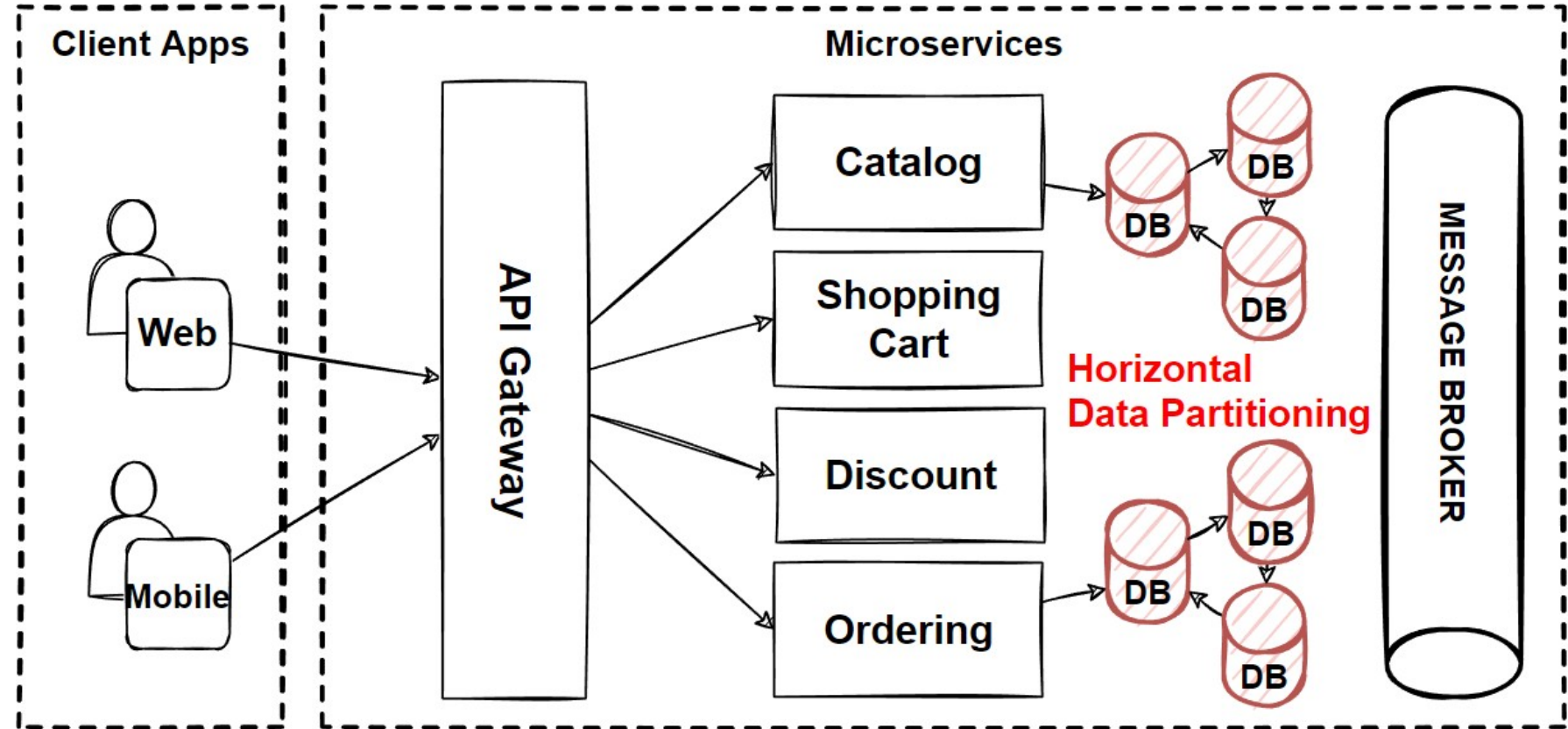
- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Message Brokers

- Kafka
- RabbitMQ
- Amazon EventBridge, SNS

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



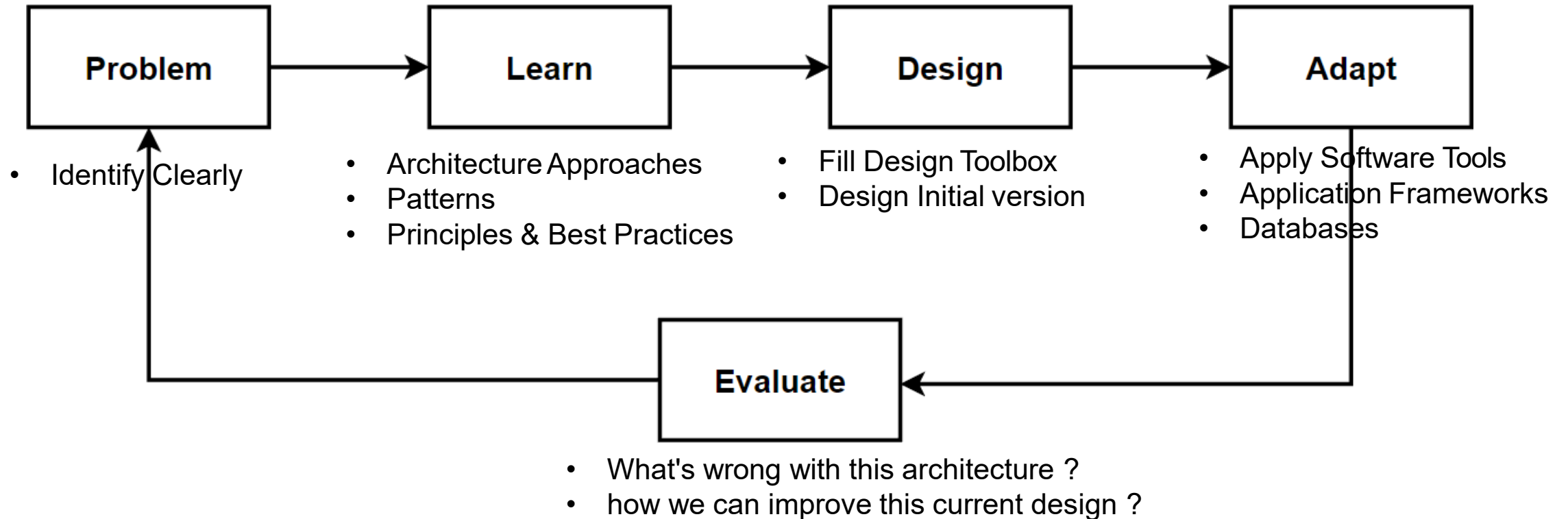
No-SQL Database

- Cassandra – NoSQL Peer-to-Peer
- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair

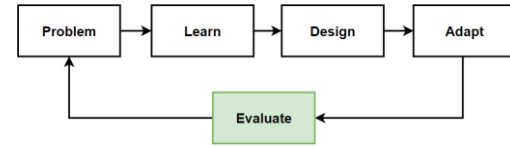
Cloud No-SQL Database

- Amazon DynamoDB
- Azure CosmosDB

Way of Learning – The Course Flow



Evaluate: Microservice Architecture with Database Sharding Pattern - Cassandra

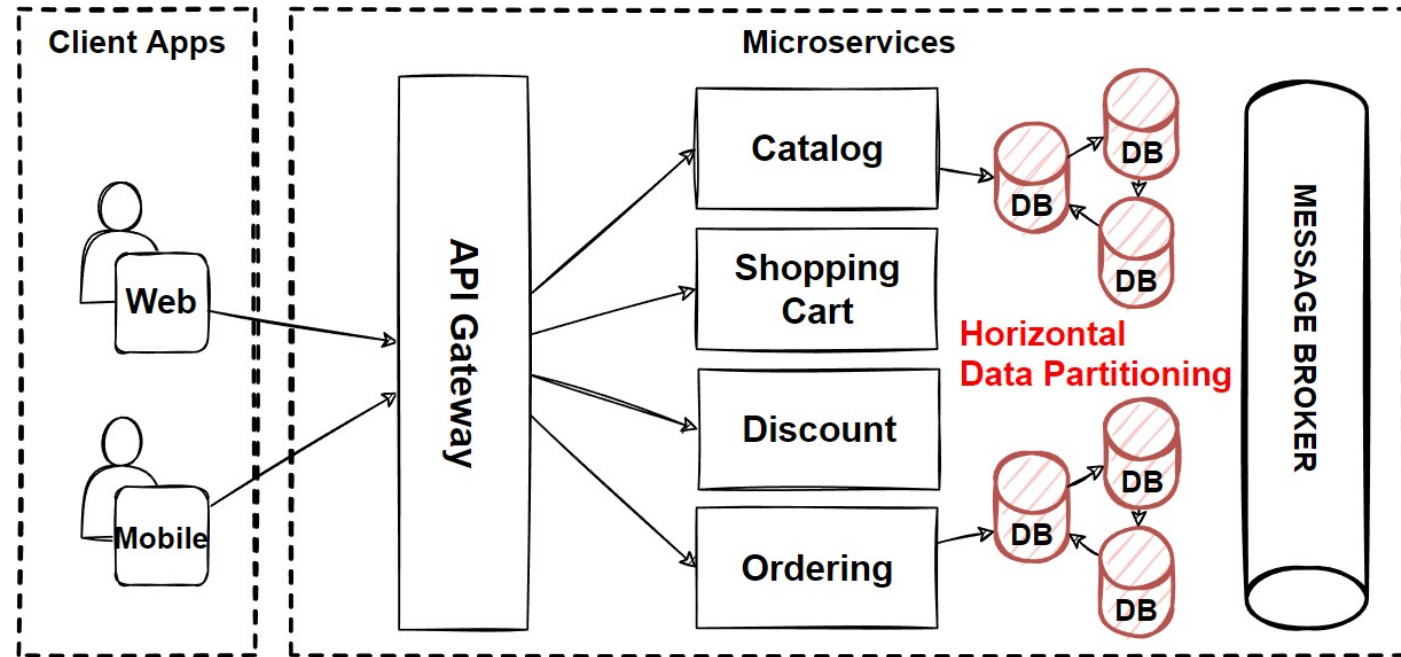


Benefits

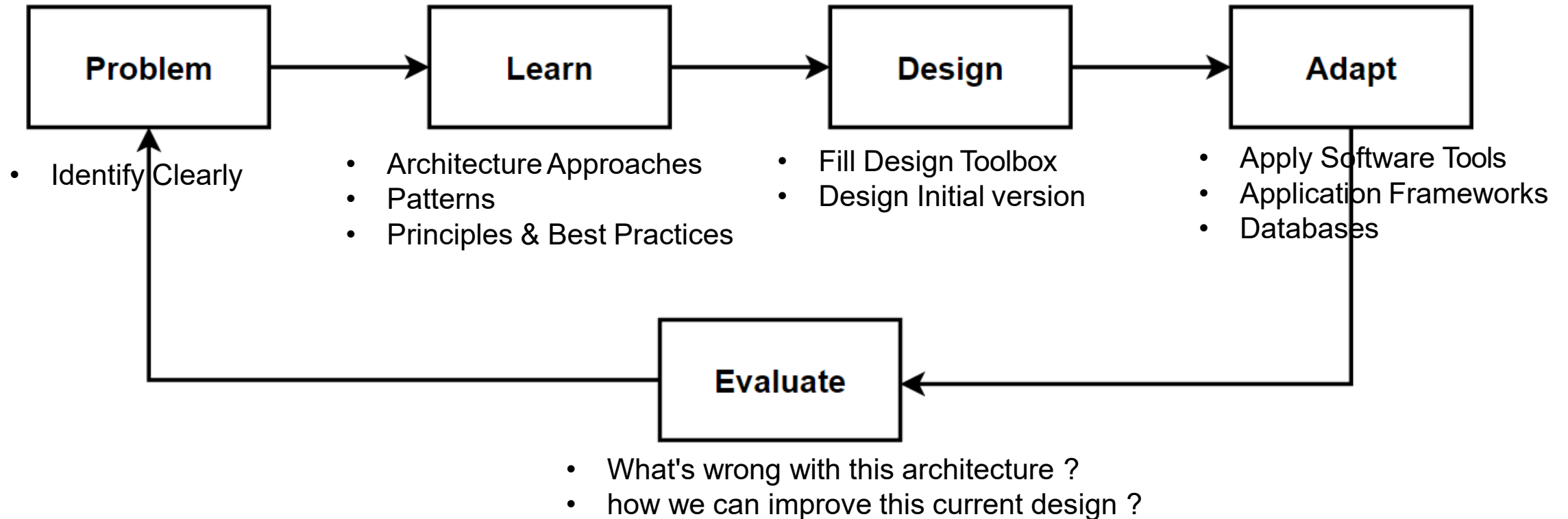
- Distributed Horizontally Scaled Databases that ready for Kubernetes Deployments
- Data Partitioning: Horizontal and Functional Data Partitioning
- Database Sharding Pattern: Increase Database Scalability, Improve Availability, Manageability
- Master-Master (Master-less) architecture with Peer-to-Peer Distributed Wide Column Database

Drawbacks

- CAP Theorem: Data Consistency, Sacrifice Strong Consistency to Partition Tolerance
- Cross-services Queries
- Complex JOIN requires operations
- Lack of Read-Write Database Separations
- Distributed Transaction Management



Way of Learning – The Course Flow



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**